



Sign & Spell VR: ISL Alphabet Practice and Sentence Construction in Virtual Reality Interim Report

TU858

BSc in Computer Science International

Student Name: Karl Negrillo

Student Number: C22386123

Supervisor: Colette Kirwan

School of Computer Science

Technological University, Dublin

Date

Abstract

The Sign & Spell VR project aims to create an immersive virtual reality application designed to facilitate the learning of the Irish Sign Language (ISL). This project addresses the need for accessible and interactive tool for sign language education, focusing specially on ISL, which is less commonly represented in existing digital learning platforms. The application utilizes the Meta Quest 3/3S virtual reality headset and the Godot 4 game engine to provide an engaging and interactive learning environment.

The core functionality of the application includes real-time tracking of user's hand movements to recognize and assess ISL alphabet letters, words, and simple sentences signed by the user. This features immediate feedback to aid learning effectiveness and user engagement. The project progresses through sequential learning modules that introduce the alphabet, enable word spelling, facilitate sentence construction, and conclude with an interactive testing phase to evaluate the user's comprehension and retention.

The final deliverable is a functional VR prototype that demonstrates the potential of virtual reality technology in language learning and accessibility. By integrating gesture recognition with VR immersion, the project offers a novel approach to learning ISL, which benefits both learners and the deaf community by promoting effective communication skills. The initiative not only supports language acquisition but also raises awareness about the cultural significance of ISL. The Sign & Spell VR project thus stands as an innovative educational tool with possibilities for further expansion to include additional sign language and more complex linguistic features.

Declaration

I hereby declare that the work described in this dissertation is, except where otherwise stated, entirely my own work and has not been submitted as an exercise for a degree at this or any other university.

Signed:

Karl Negrillo

Karl Negrillo

Date: 20/11/2025

Acknowledgements

I would like to thank my supervisor, Colette Kirwan, for her invaluable guidance and support throughout the course of this project. Her feedback and encouragement helped shape the direction of my work. I also wish to acknowledge the School of Computer Science at Technological University Dublin for providing the resources and environment necessary for completing this report.

1. Contents

1. Introduction.....	1
1.1 Project Background	1
1.2 Project Description.....	1
1.3 Project Aims and Objectives	2
1.4 Project Scope.....	3
1.5 Thesis Roadmap	3
2. Literature Review.....	5
2.1 Introduction.....	5
2.2 Alternative Existing Solutions.....	5
2.2.1 Introduction.....	5
2.2.2 ASL FingerSpeller – SideQuest	5
2.2.3 Silent Classroom VR – Meta	5
2.2.4 ASL Champ! – Science Direct.....	6
2.2.5 DEF-ISL – App Store	6
2.2.6 Design of Sign Language Learning Media Based on Virtual Reality	6
2.2.7 VR Sign Language Learning with Hand Tracking – YouTube	6
2.3 Technologies Researched.....	7
2.3.1 Meta Quest 3/3S	7
2.3.2 Godot 4 Engine	8
2.3.3 GDScript.....	8
2.3.4 JSON File Storage & Cloud Sync	8
2.3.5 OpenXR.....	9
2.3.4 Why not other Technologies for my project?	9
2.3.4.1 Why not Unity?	9
2.3.4.2 Why not Unreal Engine	10
2.3.4.3 Why not a Database?	10
2.4 Other Research.....	10
2.4.1 Ethics	10
2.4.1.1 The FATE Landscape of Sign Language AI Datasets	10
2.4.1.2 Sign Language Recognition and Interdisciplinary Challenges	10
2.4.1.3 AI Fairness for Deaf or Hard of Hearing (DHH) Users	11
2.4.1.4 WFD and WASLI Statement on Signing Avatars.....	11
2.4.1.5 Conclusion	11
2.4.2 FURPS Model.....	11

2.4.2.1 Why Use FURPS.....	12
2.4.3 Human-Computer Interaction in Sign & Spell VR	12
2.5 Existing Final Year Projects.....	13
2.5.1 Sign Language Translator (SLT)	13
2.5.2 VR Music Learner	14
2.6 Conclusions.....	14
3. System Analysis.....	16
3.1 System Overview	16
3.1.1 What is Sign & Spell VR?	16
3.1.2 How the System Works	16
3.1.3 User Experience and Interface	16
3.1.4 Practice and Feedback	16
3.1.5 Saving Progress and Accessibility.....	16
3.1.6 Inspiration and Aims	17
3.2 Requirements Gathering.....	17
3.2.1 Who Are the Stakeholders?	17
3.2.1.1 New Learners	17
3.2.1.2 Teachers and Instructors.....	17
3.2.1.3 Deaf Community	17
3.2.1.4 Potential Stakeholders	18
3.2.2 How I Collected Requirements.....	18
3.2.3 Initial Requirements List.....	18
3.3 Requirements Analysis	19
3.3.1 Organising the Requirements	19
3.3.2 Prioritising Features	19
3.3.3 Making an Initial System Model.....	20
3.3.4 Making an Initial System Model Use Case	20
3.3.5 Risk and Challenges	21
3.3.6 Reviewing Gaps	21
3.4 Initial System Specification	22
3.4.1 Initial Functional Requirements	22
3.4.2 Initial Non-Functional Requirements.....	22
3.4.3 Initial System Architecture	23
3.4.4 How it all Works	23
3.5 Conclusions.....	24

4. System Design	25
4.1 Introduction.....	25
4.1.1 Translating Requirements into Design	25
4.1.2 Design Principles and Goals	25
4.2 Software Methodology	25
4.2.1 Incremental Model.....	25
4.2.2 Advantages of Incremental Development for my Project	26
4.2.3 How Will I Apply it	26
4.3 Overview of System.....	26
4.3.1 Logical Architecture.....	26
4.3.2 Main System Modules.....	26
4.3.2.1 Data Model Class Diagram for User, Calibration and Progress.....	27
4.3.3 Physical Architecture and Development.....	28
4.3.4 Data Flow Between Modules	28
4.3.4 Modular Benefits.....	29
4.4 Design System	30
4.4.1 Applying Design Approach	30
4.4.2 Modular Design Approach	30
4.4.3 Use of UML Diagrams.....	31
4.4.4 Prototyping and Iteration.....	31
4.4.5 Scalability and Flexibility Considerations	31
4.5 Conclusions.....	31
5. Testing and Evaluation	32
5.1 Introduction.....	32
5.2 Plan for Testing.....	32
5.3 Plan for Evaluation	33
5.4 Conclusions.....	34
6. System Prototype	35
6.1 Introduction.....	35
6.2 Prototype Development.....	36
6.2.1 Hand Pose Detector Setup	36
6.2.2 Signal Handlers for Pose Detection.....	37
6.2.3 Mapping Hand Poses to ISL Letters.....	38
6.2.4 Updating UI Feedback	38
6.2.5 Using Godot Addons, Environment Setup, and Hierarchy.....	38

6.3 Results	40
6.4 Evaluation.....	43
6.4.1 Hand Tracking Accuracy	43
6.4.2 Feedback and UI.....	44
6.4.3 User Testing.....	44
6.4.4 Plans for Further Testing	44
6.5 Conclusions.....	44
7. Issues and Future Work	45
7.1 Introduction.....	45
7.2 Issues and Risks	45
7.3 Plans and Future Work.....	45
7.3.1 Project Plan with GANTT Chart.....	46
References.....	47
A) Appendix A: System Model and Analysis.....	A-1
B) Appendix B: Design.....	B-1
C) Appendix C: Prompts Used with ChatGPT	C-1
D) Appendix D: Additional Code Samples.....	D-1

1. Introduction

Irish Sign Language (ISL) plays a vital role in accessible communication and inclusion in Ireland. With the increasing demand for interactive and engaging learning tools, this project explores the use of virtual reality to enhance ISL education, making it more practical and user-friendly for beginners.

1.1 Project Background

Sign Language is the primary means of communication for over 70 million Deaf People worldwide, yet many learners struggle to find effective, accessible resources to master it independently. Learning sign language is important for communication and inclusion, but it can be difficult for beginners to learn on their own. Many people use books, online videos, or take ISL classes from the Irish Deaf Society. While classes provide feedback, books and videos don't correct mistakes. The app ASL Fingerspeller on SideQuest notes that VR and hand tracking make learning "more immersive and interactive," showing how new technology can make signing more engaging [1]. VR allows users to move in a 3D environment, helping them better understand hand shapes and motion.

This project is also important because there are very few tools for learning Irish Sign Language (ISL). Most existing apps focus on American Sign Language (ASL), which uses one hand. ISL uses two hands and different grammar rules, so it needs its own learning tools. A VR-based ISL system could make learning more accessible and inclusive for the Deaf community in Ireland.

Some VR apps already teach sign language, but mostly ASL. For example, ASL Fingerspeller [1] lets users spell words in VR, and Silent Classroom VR [2] offers a classroom-style setup. ASL Champ! [3] uses AI for real-time feedback, but these apps don't include sentence practice or ISL content.

In Ireland, DEF-ISL [4] and CrowdSign exist, but they mainly show static videos or pictures with no interaction. The study Design of Sign Language Learning Media Based on Virtual Reality developed an interactive VR sign language app with visual feedback and found users gained confidence after using it [5].

Since this project uses Godot, features like OpenXR or controller gestures can be handled through rule-based detection instead of machine learning. Novaliendry et al. [5] describe using "a software testing technique that focuses on the functionality of the software without considering its internal structure or implementation details," which suits early-stage testing in Godot. Research also shows that VR learners perform better with "immediate and interactive feedback" [3]. The video VR Sign Language Learning with Hand Tracking [6] demonstrates how visual feedback helps users correct signs in real time. Together, these findings suggest a rule-based, feedback-driven VR system is both feasible and effective for teaching ISL.

1.2 Project Description

What should this immersive VR Sign Language assistant do?

The goal of this project is to help users learn Irish Sign Language (ISL) in an immersive, hands-on way similar to real-world practice. Using the Meta Quest 3/3S VR headset, the application will allow users to engage with interactive lessons and receive feedback on their signing performance. Learners will practice signing the 3D hand shapes of the ISL alphabet, common words that deaf people use, and phrases. To provide accurate recognition tailored to each user, the system will require an initial configuration phase where users create an account and calibrate their hand movements. The hand tracking data collected during this configuration will be stored in a JSON file, which serves as a database of user-specific hand coordinates for future sign recognition and analysis.

Users will begin with learning the alphabet, then progress to signing common words and phrases. After practice, users can evaluate their learning through an interactive testing section. The system allows users to revisit and reinforce any learning modules as needed to improve proficiency.

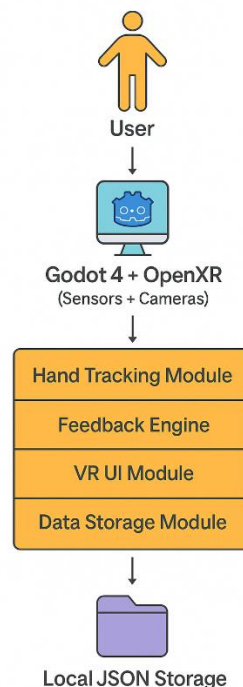


Figure 1: System Architecture Diagram for Sign & Spell VR

1.3 Project Aims and Objectives

To design and implement an interactive virtual reality application that enables users to learn and practice Irish Sign Language (ISL) efficiently through immersive lessons, personalized hand tracking, and real-time feedback.

Objectives:

- Develop a VR application for Meta Quest 3/3S using Godot 4
- Integrate Hand-Tracking

- Implement account creation and user-specific hand configuration system, with data saved in JSON format
- Portray and Create the ISL alphabet
- Create a learning space for common words and phrases in ISL taken from users.
- Develop a testing/assessment section to evaluate user progress
- Allow users to revisit and practice previously learned content as needed.
- Gather user feedback to refine usability and overall learning experience

1.4 Project Scope

The project scope focuses on the VR application. It clarifies the specific features and deliverables that will be developed, as well as what falls outside the project's responsibilities. By setting clear limits, the project scope ensures that work remains targeted, achievable within available resources and time, and helps manage expectations for stakeholders and users.

In Scope:

- **ISL Learning Modules** – Users practice the ISL alphabet, common words, and simple phrases in an interactive VR Environment
- **User Registration and Configuration** – Users create accounts and calibrate their hand configuration from the alphabet, which is then saved to a JSON database for tailored sign recognition.
- **Real-Hand Tracking & Feedback** – The system tracks hand movements and provides immediate visual feedback to guide users and correct errors.
- **Progress Assessment** – Users can test their knowledge and review feedback to evaluate learning and identify areas of improvement
- **Intuitive VR Interface** – The application is designed for the Meta Quest 3/3S headset, ensuring a comfortable and accessible user experience
- **Single-User Mode** – The learning experience is focused on individual progress, with users able to repeat modules and revisit lessons as needed.

Out of Scope:

- Development for VR platforms other than Meta Quest 3/3S
- Cloud Database integration (data stored locally in JSON)

1.5 Thesis Roadmap

This report is structured to provide a comprehensive overview of my project Sign & Spell VR. The following chapters are included:

- **Literature Review** – This chapter will explore existing solutions, relevant technologies, and related research, providing an evaluation that informs the project direction.
- **System Analysis** – This chapter will outline the system requirements, including gathering and analysing user needs, and specifying system objectives

- System Design – In this chapter, I will be writing about the selected software development methodology, system architecture, and user interface design.
- Testing and Evaluation – This chapter will cover the testing strategies I used, user evaluation, and results that assess the system's usability and effectiveness.
- Issues and Future Work – Finally this chapter reflects on the challenges faced during the project and proposed direction for future improvements.

2. Literature Review

2.1 Introduction

This chapter presents the research and exploration carried out to inform the development of the project. A range of existing solutions related to sign language education and learning technology is analysed, evaluating their strengths and limitations. The chapter outlines and compares technologies considered for use in Sign & Spell VR, focusing on options for core features such as hand tracking, user interaction, and immersive feedback. Further research is discussed to understand key factors that shape accessible learning, with particular attention given to community needs and technical feasibility. Previous final year projects relevant to this topic are also reviewed, illustrating how their findings influenced the overall direction of the project and the decisions made throughout its lifecycle.

2.2 Alternative Existing Solutions

2.2.1 Introduction

There are now many sign language apps available, and they all approach teaching in slightly different ways. To compare them in a fair and structured manner, the FURPS model was selected, which stands for Functionality, Usability, Reliability, Performance, and Supportability. This framework provides a simple way to identify which apps work well and where they may be lacking. For this project, ASL Fingerspeller, Silent Classroom VR, ASL Champ, DEF-ISL, and a YouTube video on VR sign language learning with hand tracking are examined. Each one is considered in terms of how it supports sign language learning, what works well, what could be improved, and how it relates to the goals of Sign & Spell VR.

2.2.2 ASL FingerSpeller – SideQuest

ASL Fingerspeller is a VR app on SideQuest that allows users to practise fingerspelling in American Sign Language on Meta Quest. Its store description states that users can “practice spelling names, places, and random words”, and includes a timed mode to add challenge. User comments highlight that it is helpful for learning the ASL alphabet quickly but can become repetitive over time. This shows strong Functionality (alphabet practice with random words) but limited Variety and Progression. For Sign & Spell VR, this suggests adding structured challenge elements, such as quiz modes, while also avoiding repetition by gradually expanding from alphabet practice to everyday words and simple sentences. [1]

2.2.3 Silent Classroom VR – Meta

Silent Classroom VR is an educational experience for learning American Sign Language in a simulated classroom setting on Meta Quest. The store description invites users to “Set in a virtual ASL classroom, players engage with hand-tracking technology to sign alphabets, reinforcing their finger-spelling skills in a fun and educational way”, highlighting its focus on realism and presence. Reviews on the page frequently mention that it feels like being in an actual class but in VR, which shows strong immersion and usability for learners who like structured lessons. However, the content is centred on ASL and a classroom scenario, rather

than short practice activities or support for other sign languages. For Sign & Spell VR, this example demonstrates how an immersive setting can make practice feel more authentic, but the project instead aims for a simpler, more focused environment tailored to ISL beginners, with quick practice loops and immediate feedback rather than a full classroom simulation. [2]

2.2.4 ASL Champ! – Science Direct

ASL Champ! is a VR game that uses AI-based sign recognition to provide learners with feedback. The ScienceDirect abstract notes that it “provides real-time feedback about whether the produced sign is correct”. This directly supports the Functionality and Usability dimensions in FURPS by showing that immediate, precise feedback improves both learning outcomes and motivation. Sign & Spell VR does not use AI in the current phase, but this research motivates a rule-based real-time feedback system: even without machine learning, the project aims to give instant visual responses (correct/incorrect and hints) to keep users engaged in a similar way to ASL Champ! [3]

2.2.5 DEF-ISL – App Store

DEF-ISL is mainly for Indian Sign Language, but it’s still a helpful example. The developers say it “makes learning easy through Indian Sign Language,” and users say it’s good for all ages and easy to pick up. It looks like it has a lot of signs and videos which reinforces the project’s goal of going beyond static resources by adding active practice, tracking, and feedback tailored specifically to ISL. [4]

2.2.6 Design of Sign Language Learning Media Based on Virtual Reality

The paper by Novaliendry et al. describes the design of a VR-based sign language learning application aimed at students with special needs. The application runs on affordable VR devices and combines interactive 3D environments with visual and audio feedback to support learning. The authors report that “users feel involved in the VR experience and experience increased confidence in using sign language after using this application,” showing that immersion and feedback can significantly improve both engagement and confidence in sign language use.

This finding is directly relevant to Sign & Spell VR. It supports the decision to prioritise clear, immediate feedback for each attempted sign and to use an immersive VR environment rather than static media. By adopting similar principles—interactive practice, visual feedback, and simple, confidence-building activities—Sign & Spell VR aims to provide ISL learners with the same kind of engaging experience, but tailored specifically to Irish Sign Language and the Meta Quest 3 platform. [5]

2.2.7 VR Sign Language Learning with Hand Tracking – YouTube

The YouTube video “VR Sign Language Learning with Hand Tracking” demonstrates how Meta Quest hand tracking can be used without controllers. The creators state that “by enabling people to use their real hands in VR, hand tracking opens up many new capabilities and use cases”. This supports the choice of a controller-free interaction model for Sign &

Spell VR and reinforces the focus on natural, hand-based input rather than abstract controller buttons. The project builds on this idea by applying hand tracking specifically to ISL alphabet practice and calibration, aiming to create a more intuitive and accessible learning experience for users who may be new to both sign language and VR. [6]

2.3 Technologies Researched

For this project, several technologies were investigated to determine which would be most suitable for building a VR sign language learning application. Popular engines such as Unity and Unreal Engine were considered, but the Godot 4 Engine was ultimately selected because it is free, open source, and widely regarded as approachable for developers with limited game-development experience. It is also increasingly popular among indie developers, and there is a growing set of tutorials specifically focused on VR development in Godot. In addition, Godot 4 aligns with the XR Prototyping module used in the course, which makes it a practical choice for both learning and implementation. [7]

2.3.1 Meta Quest 3/3S

The Meta Quest 3 headset was chosen as the target platform. It functions as a standalone device and does not require a high-end gaming PC, which simplifies setup and makes it more accessible for classroom or home use. Crucially, the Quest 3 supports built-in hand tracking, allowing accurate capture of finger movements without controllers. This is important for a sign language learning app because it enables more precise and natural representation of ISL hand shapes. The wireless design allows users to move freely while practising, reducing complexity in the overall system.

A substantial number of guides and examples exist for developing educational and training applications on Meta Quest, which is helpful given the learning curve of VR development. The device is also increasingly used in educational settings, reinforcing its suitability as a platform for Sign & Spell VR. [14]



Figure 2: Meta Quest 3/3S

2.3.2 Godot 4 Engine

The project uses the Godot 4 game engine as its primary development environment. Godot is free and open source, removing licensing barriers and making it suitable for a student project. Although it is less widely used than Unity or Unreal, recent versions include significantly improved support for VR through OpenXR, which is essential for Meta Quest integration.

Godot 4 supports full 3D environments and provides tools that are well-suited to building educational and experimental projects. Because the project centres on creating an environment where users can practise ISL and receive feedback, Godot’s scene system, scripting model, and VR plugins are sufficient to implement the required features. Its open-source nature also means there is an active community producing tutorials, examples, and answers to common problems—an important factor when learning new XR techniques. [8]

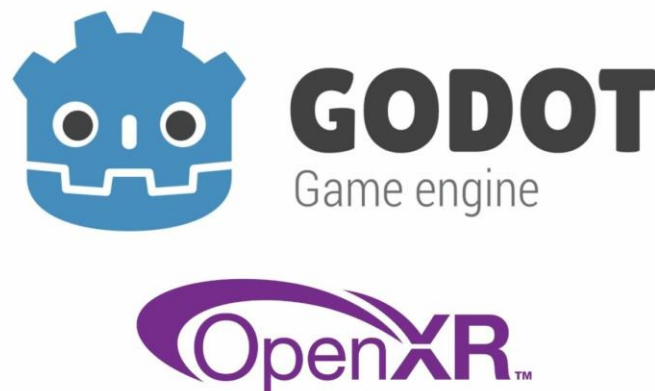


Figure 3: Godot and OpenXR

2.3.3 GDScript

Most of the project’s logic is implemented in GDScript, Godot’s built-in scripting language. GDScript has a syntax similar to Python, which makes it easier to learn and read, especially for developers with some prior Python experience. Alternative languages such as C# are supported, but they typically require more configuration and are less tightly integrated into the engine’s workflow.

Using GDScript allows rapid iteration and testing when implementing features such as hand-movement checks, correctness feedback, and saving user progress. This reduces overhead when experimenting with new ideas or fixing bugs, which is particularly valuable in a project that involves frequent changes to interaction logic and UI behaviour. [8]

2.3.4 JSON File Storage & Cloud Sync

For storing calibration data and user information, the project uses JSON files rather than a full external database. JSON is directly supported in Godot, and the engine exposes a FileAccess API alongside the user:// path for reading and writing persistent data without additional plug-ins. Each learner’s hand calibration and progress can be stored in a separate

JSON file and loaded when the application starts, ensuring that personalised settings are restored automatically.

In future work, the same JSON structures could be sent to a cloud service (for example Firebase or a custom backend) using HTTP requests, enabling cross-device synchronisation of calibration and progress. This approach is less complex than deploying a full client-server database such as PostgreSQL or MongoDB, but still provides a clear evolution path if multi-device access becomes a requirement. The choice is consistent with OpenXR and Godot XR documentation, which leave persistence responsibilities to the application: calibration and progress are therefore stored via FileAccess in the user:// path, with the option to forward the same data to the cloud later. [13]

2.3.5 OpenXR

The project uses OpenXR with Godot 4 as the foundation for VR support on Meta Quest 3. OpenXR is an open standard that defines how VR hardware and applications communicate, allowing the application to work more smoothly across compatible devices. Godot 4 includes native OpenXR support, so activating the plugin and following available guides is sufficient to enable core VR features.

OpenXR provides access to essential capabilities such as head tracking and hand tracking without requiring custom low-level integrations. This allows the hand-tracking module in Sign & Spell VR to obtain real sensor data from the Quest and feed it directly into the sign-recognition logic during practice sessions. It also contributes to a smoother, less glitchy VR experience, which is important for beginners who may be sensitive to latency or instability. By following the official documentation and examples, VR rendering and tracking could be enabled relatively quickly, allowing more development time to focus on ISL-specific features rather than platform integration. [8]

2.3.4 Why not other Technologies for my project?

2.3.4.1 Why not Unity?

Unity was considered as a potential engine because it is widely used for VR and offers many built-in features and assets. However, the overall setup was judged to be less beginner-friendly, especially for quickly enabling hand tracking and OpenXR-based VR. The configuration process can be complex, with many options and systems that are not directly relevant to this project's scope. Unity's licensing model can also introduce limits or costs when using certain features, which is less suitable for a student project.

Given that the associated XR Prototyping course focuses specifically on Godot for VR and hand tracking, it was more efficient and consistent to use the same engine in this project. This alignment avoids the need to learn a completely new toolchain and allows development time to be spent implementing and testing features rather than managing engine-specific setup.

2.3.4.2 Why not Unreal Engine

Unreal Engine is highly capable, particularly for high-end games and graphically intensive experiences. However, most available guidance and examples are aimed at large projects or experienced developers. For a smaller educational prototype, Unreal would introduce unnecessary complexity, focusing more on advanced rendering and less on rapid iteration for hand-tracking interactions. As a result, it was considered excessive for the needs of Sign & Spell VR, where the emphasis is on functionality, feedback, and accessibility rather than cutting-edge graphics.

2.3.4.3 Why not a Database?

Relational or cloud databases (such as SQL or Firebase) were also considered for storing user accounts and calibration information. After analysis, this approach was deemed more complex than necessary for the expected scale of the application. The system is not designed for thousands of concurrent users; instead, it primarily needs to store configuration and progress data for individual learners on their own devices.

JSON files integrate cleanly with Godot's FileAccess API, require minimal setup, and are easy to inspect and debug during development. This keeps the architecture simple while satisfying the requirement for persistent local storage. If the project later expands to support multi-user scenarios or centralised analytics, the JSON data model can still be migrated to a database-backed solution, but for the current prototype, file-based storage is the most practical choice.

2.4 Other Research

2.4.1 Ethics

2.4.1.1 The FATE Landscape of Sign Language AI Datasets

This paper highlights critical ethical concerns around sign language AI datasets, focusing on fairness, accountability, transparency, and ethics (FATE). The authors point out that “signing videos are deeply personal and culturally specific data,” requiring careful consideration of consent and control by Deaf individuals. They emphasize the importance of involving Deaf communities “to avoid cultural appropriation and to build trust.” They also talk about challenges in creating datasets that are both representative and respectful to the linguistic diversity of sign languages. [9]

2.4.1.2 Sign Language Recognition and Interdisciplinary Challenges

This study underscores the necessity of cross-disciplinary collaboration in sign language AI research. It stresses ethical issues where “technologists often develop systems without Deaf community input, leading to poor quality and mistrust”. The paper argues that sign languages are complex and culturally rich, stating, “Technologies must respect linguistic identity, or risk producing harmful misrepresentations.” The involvement of linguists, computer vision experts, and Deaf scholars is essential to produce usable and ethical technologies. [10]

2.4.1.3 AI Fairness for Deaf or Hard of Hearing (DHH) Users

The authors address fairness issues in AI systems designed for DHH users, noting that many models “perform poorly with Deaf users due to limited and biased data”. They stress that lack of transparency and interpretability in AI models creates challenges for accessibility. The study calls for “developing trustworthy AI tools with inclusive data and evaluation metrics designed for DHH needs,” emphasizing researchers’ ethical responsibility toward marginalized users. [11]

2.4.1.4 WFD and WASLI Statement on Signing Avatars

The World Federation of the Deaf (WFD) and the World Association of Sign Language Interpreters (WASLI) caution that signing avatars are not a substitute for human interpreters. They state, “Avatars cannot replicate the complex grammar and cultural nuances of sign languages”. For critical communications, human interpreters remain essential. The statement advises, “Use of avatars should be guided by Deaf community consultation and limited to non-urgent content.” [12]

2.4.1.5 Conclusion

These papers collectively highlight that ethical sign language AI development requires respecting Deaf culture, ensuring fairness through community involvement, and maintaining transparency in technology. Sign & Spell VR, as an educational VR application, must consider these principles carefully. Incorporating Deaf feedback and providing clear, accurate sign representations aligns with the recommendation that technologies support rather than replace human expertise. In addition, given the limitations of AI and avatars noted in these works, the project focuses on providing accessible, immersive learning with real-time feedback, rather than automated interpretation, to empower users while respecting the linguistic complexity and cultural significance of sign languages.

2.4.2 FURPS Model

The FURPS model is a framework developed at Hewlett-Packard to help evaluate and organize software requirements and quality attributes. The acronym FURPS stands for:

- **Functionality:** The features, capabilities, and security of the software. This covers what the system actually does, what functions it performs, and whether it meets user or project needs. In sign language learning apps, functionality could be about how well it recognizes signs, supports quizzes, or provides helpful feedback.
- **Usability:** How easy and intuitive the software is for real people to use. This includes user interface design, consistency, documentation, and how quickly a new user can learn to operate the system. For educational apps, usability is about making sure learners feel comfortable and can navigate without getting lost or frustrated.
- **Reliability:** The likelihood that the software runs smoothly without crashing or making mistakes, and how well it recovers from errors. It also includes uptime, error rates, and how predictable the system is. For example, learning apps should not freeze, lose user progress, or misunderstand simple inputs.

- **Performance:** How fast the software responds, how efficiently it uses resources, and its ability to handle many users or complex tasks. In a VR sign language app, this would mean smooth hand tracking, minimal lag, and no stuttering, even during busy lessons.
- **Supportability:** How easily the software can be maintained, updated, debugged, or enhanced later. This includes aspects like testability, adaptability, installing updates, and support for localization or different devices. For a student project, good supportability means that new features can be added or bugs fixed without starting over.



Figure 4: FURPS Model

2.4.2.1 Why Use FURPS

The main reason this project uses the FURPS model is to ensure that all important qualities of the software are considered, not just features but also usability, robustness, and maintainability. It's a useful checklist when comparing different technologies or solutions and helps ensure nothing major is forgotten in the design and review stages.

2.4.3 Human-Computer Interaction in Sign & Spell VR

The design of Sign & Spell VR incorporates widely recognized HCI principles such as Nielsen's Usability Heuristics, which include guidelines for system feedback, user control, consistency, error prevention, and simplicity. For example, providing immediate feedback after users perform signs aligns with Nielsen's emphasis on visibility of system status, helping learners know if their sign was correct or needs adjustment.

Other key principles come from Schneiderman's Eight Golden Rules of Interface Design, which stress consistency, error prevention, and minimizing memory load, all relevant to VR environments where users must focus on learning and performing gestures naturally. By reducing cognitive load through simple and predictable navigation, the application supports these golden rules. [15]

Additionally, accessibility standards such as the Web Content Accessibility Guidelines (WCAG) influence the design to ensure inclusivity for users with different abilities, such as left-handed signers or VR novices. These foundations provide a structured approach to enhancing usability and engagement in the virtual learning experience. [16]

2.5 Existing Final Year Projects

2.5.1 Sign Language Translator (SLT)

Title: Sign Language Translator (SLT)

Student: Andrei Botnari

Description (brief): This project is about creating a system that can translate Irish Sign Language (ISL) gestures into text using the Xbox Kinect Sensor. This student developed a recognition pipeline that captured depth data, segmented the hands from the background, and matched the input against predefined gestures. From this project, the aim is to evaluate the accuracy the Kinect sensor under different environmental conditions such as lighting, distance, and motion.

What is complex in this project: The real time gestures recognition using low-resolution depth data. From the project, ISL contains many hand shapes that appear visually similar, making classification and tracking difficult. Managing environmental interference and differentiating between similar gestures (e.g., E, S, and T) required careful calibration and algorithm tuning.

What technical architecture was used: The student used the Kinect V2 depth sensor connected to a PC and developed in Python and C#. This project followed the Spiral Development model, which emphasises the iterative testing of gesture recognition accuracy and usability. The architecture included modules like image capture, segmentation, and features extraction before translating gestures to text output.

Explain key strengths and weaknesses of this project, as you see it:

Strengths: Hard focus on experimental validation, detailed testing across varied conditions. The spiral model ensured improvements. It provided an early exploration into ISL recognition, which remains a niche and research area.

Weaknesses: This project depended heavy on the Kinect hardware, which is now outdated and sensitive to lighting and distance as the resolution is low. It lacked interactive feedback, which means that users could not correct or practice signs dynamically, which limits learning effectiveness. [18]

2.5.2 VR Music Learner

Title: VR Music Learner

Student: Maciej Golubski

Description (brief): This student developed a virtual reality environment where users could learn musical instruments such as guitar or piano in an immersive, game-like setting. He integrated VR Gameplay, sound recognition, and a connected mobile app to track user progress and statistics. It aimed to make music learning more engaging through interaction, rhythm matching, and score tracking.

What is complex in this project: The challenge I have read from this project is the VR motion input, audio recognition, and mobile data communication. Maintaining accurate timing between the player's performance and in-game feedback required strong optimization.

What technical architecture was used: This project is developed in Unity using C#, and using the Oculus Quest Headset for VR interaction, a web server and database for the scoring system, and a mobile companion app for data visuality. The student used Agile/Scrum workflow for iterative development and user testing.

Explain key strengths and weaknesses of this project, as you see it:

Strengths: Good integration with VR, mobile, backend technology. The focus is strongly seen on gamifying it and feedback loops that enhance learning and motivation. This shows well-thought user experience for the user.

Weaknesses: It has limited educational depth. Mainly focuses on music theory. System performance may change depending on network and the accuracy of audio input. The student could have designed it from a broader perspective of testing and adding additional learning elements. [17]

2.6 Conclusions

Overall, the research and reviews presented in this chapter have been valuable in shaping the direction and design of Sign & Spell VR. Examining existing solutions and apps for sign language education, such as ASL Fingerspeller, Silent Classroom VR, ASL Champ!, DEF-ISL, and other research projects and videos, showed that each has a different approach to digital sign language learning. Most of these tools focus on American or Indian Sign Language, which means there is relatively little support specifically for Irish Sign Language; this gap is one of the motivations for Sign & Spell VR.

The FURPS model helped organise each existing solution in terms of strengths and areas for improvement. Features such as real-time feedback, as seen in ASL Champ!, and immersive environments, as in Silent Classroom VR, indicate that learning is more effective when users receive instant responses and feel present in a realistic setting. These findings informed the decision to keep Sign & Spell VR simple for beginners while still providing useful feedback and motivation for repeated practice. Educational VR research emphasising visual feedback also reinforces the importance of clear, immediate responses to increase learner confidence and encourage ongoing engagement.

When investigating technologies, different platforms and tools were compared before selecting Godot 4, Meta Quest 3, GDScript, and JSON file storage. Godot 4 fits well with the developer's skill level and the XR Prototyping module, which specifically covers VR and hand tracking in this engine. Meta Quest 3 was chosen not only for its portability, but also for its finger-tracking capabilities and growing use in education. GDScript simplifies development and iteration, while JSON-based storage keeps calibration and user data handling lightweight and manageable, with the option of cloud syncing in the future.

The ethical and community aspects of sign language technology, drawn from recent academic papers and official statements, emphasise that projects of this kind must involve Deaf communities, respect the complexity and culture of sign languages, and make learning resources fair and accessible. This implies that Sign & Spell VR should not attempt to replace human interpreters, and should remain open to feedback from ISL users to ensure that signs are represented accurately and respectfully.

Reflecting on previous final year projects, such as Sign Language Translator and VR Music Learner, highlights how hardware choices, feedback mechanisms, and engagement strategies influence both the strengths and limitations of a system. These examples reinforce that selecting appropriate tools, providing real-time feedback, and designing for genuine user needs—rather than purely technical novelty—are critical to success.

Taken together, this chapter's research has guided informed decisions for Sign & Spell VR. The project aims to provide an intuitive interface for beginners, deliver real-time feedback to support learning, and remain accessible on relatively affordable technology. It is not intended to solve every challenge in ISL education, but rather to provide a practical, ethical VR learning tool that addresses an identified gap and can continue to evolve based on both technological progress and community input.

3. System Analysis

3.1 System Overview

3.1.1 What is Sign & Spell VR?

Sign & Spell VR is a virtual reality application designed to help users learn the basics of Irish Sign Language (ISL). It targets beginners, such as students or anyone interested in ISL, who want to practise either at home or in educational settings. The aim is to make sign language learning more accessible and engaging by providing interactive practice with virtual hands in a VR environment rather than relying solely on passive video content.

3.1.2 How the System Works

When a user puts on the Meta Quest 3 headset, they are placed in a simple, uncluttered virtual room. The system uses the headset's cameras and sensors to track the user's hands in real time and render virtual hands that mirror their movements. Users can then practise forming ISL letters, words, and, in later stages, short sentences, while the system provides immediate feedback on how closely their signs match the expected shapes.

3.1.3 User Experience and Interface

From a user-experience perspective, the application is intended to be straightforward and easy to navigate. Menus provide clear options such as "Calibrate Hands", "Practice Letters", "Practice Words", and "View Progress". A short tutorial introduces the main interactions on first use, and contextual help tips are available when users appear stuck or unsure. The interface is designed to avoid overwhelming learners with too many simultaneous choices, and all navigation occurs in VR using simple gestures or large, easily selectable buttons.

3.1.4 Practice and Feedback

Each practice session begins with a calibration step to maximise hand-tracking accuracy. After calibration, users choose which aspect of ISL they wish to practise. When a sign is recognised correctly, the system displays positive feedback (for example, a "Well done!" message); when it is not, the system encourages another attempt or provides a brief hint. This loop of attempt, feedback, and retry is central to supporting incremental improvement.

3.1.5 Saving Progress and Accessibility

The system is designed to automatically save calibration data and user progress so that learners can resume from where they left off in later sessions, and eventually even after switching devices once cloud synchronisation is introduced. Accessibility is treated as a core requirement: the interface is intended to support left-handed users and those unfamiliar with VR. To this end, the design avoids very small targets or complex controls that might cause confusion or discomfort.

3.1.6 Inspiration and Aims

The overall layout and interaction approach are informed by the initial project proposal and the literature review, which highlighted the importance of real-time feedback, beginner-friendly interfaces, and self-paced learning. Sign & Spell VR draws inspiration from existing interactive VR learning apps but adapts their ideas specifically to ISL, aiming to provide a simpler, more focused experience that makes learning Irish Sign Language both approachable and enjoyable.

3.2 Requirements Gathering

3.2.1 Who Are the Stakeholders?

Identifying the main stakeholders for this project is important because it guides which features are prioritised and how the application is designed. Different stakeholder groups have different needs and expectations, so the analysis considers not only end users but also those who may support, use, or be affected by the system in other ways.

3.2.1.1 New Learners

These are mostly people starting out with Irish Sign Language and virtual reality. Their main goal is to learn ISL in a way that feels approachable and supportive.

Persona: Sophie

Sophie is a college student who's heard about ISL but has never tried learning it before. She also hasn't used a VR headset, so everything feels a bit new and maybe intimidating. Sophie hopes that the app will give her clear instructions, let her go at her own pace, and provide lots of encouragement if she gets signs wrong. If things get too technical or confusing, Sophie might lose motivation, so keeping things simple is key.

3.2.1.2 Teachers and Instructors

Teachers will want to use the app as a learning aid in lessons or for assigning practice at home. Their needs include easy setup, management of different users, and being able to monitor or support their students' learning.

Persona: Martin

Martin runs ISL classes for adults. He wants a tool that's quick to set up with minimal fuss, so students can jump straight into practicing. Martin likes being able to see who needs extra help and would appreciate ways to check progress after each lesson. He also values any notes in the app that help him explain features to students, especially those who aren't tech-savvy.

3.2.1.3 Deaf Community

Members of the Deaf community (ISL users, interpreters, advocates) help ensure the app is respectful, accurate, and culturally sensitive.

Persona: Aisling

Aisling is an ISL interpreter and strong advocate for Deaf awareness. She's keen to see more

tools making ISL accessible but is cautious about apps teaching incorrect signs or oversimplifying the language. She's happy to offer feedback if asked and hopes the app developers listen to her suggestions, particularly about sign accuracy and cultural respect.

3.2.1.4 Potential Stakeholders

In the future, more people might use the app, like families, researchers, or schools. Being aware of this early on helps keep the design open for improvements and wider use.

Persona: Mrs. O'Connor

Mrs. O'Connor has a Deaf son and wants tools that help their family learn ISL together. She values apps that are gentle for beginners, easy to follow, and work across different devices. She's especially happy if an app lets her review lessons at her own pace after the kids have gone to bed.

3.2.2 How I Collected Requirements

To determine which features and functions the ISL VR application should include, the project used a combination of research, user feedback, and practical experimentation. Existing sign language learning applications and academic papers were reviewed to identify effective design patterns, especially in VR-based learning. Particular attention was paid to user reviews and forum discussions, where learners often describe which aspects of an app are frustrating and which features they find most helpful.

Informal conversations were held with classmates and friends interested in learning sign language, as well as with Deaf acquaintances, to understand what they would expect from a VR learning tool. The plan for later stages is to use simple questionnaires or online forms once a working prototype is available, so users can give structured feedback on usability, clarity, and missing features.

Experience gained from the XR Prototyping course also feeds into the requirement-gathering process. Notes are kept whenever development challenges arise or when particular interaction patterns prove especially helpful, so these insights can be reflected in the evolving requirements for Sign & Spell VR.

3.2.3 Initial Requirements List

Based on the review of existing applications, research literature, stakeholder feedback, and early experiments with the prototype, the following initial requirements were identified as the core needs for Sign & Spell VR:

- The app must use Meta Quest 3's hand tracking accurately, so users don't need controllers.
- There should be an easy-to-follow calibration system, so all users get good results, no matter their hand size or starting position.
- The user interface must be clear and simple, using big buttons and straightforward instructions.
- Each time a user makes a sign, the app should give instant feedback, either confirming it was correct or suggesting what to fix.

- Progress and calibration settings must be saved, so users don't have to start over every session.
- If possible, there should be an option to sync progress online, so users can keep track of their learning even if they switch devices.
- The app must run smoothly on Meta Quest 3 hardware, with no crashes or big slowdowns.
- The system should be built in Godot 4 (since that's what my XR course covers), using GDScript and JSON files for simplicity.
- Accessibility should be considered, making the app usable for beginners and possible left-handed users.

3.3 Requirements Analysis

3.3.1 Organising the Requirements

After the requirements were gathered, they were organised to keep the scope clear and manageable. First, they were divided into two categories. Functional requirements describe what the application must do for the user, such as recognising hand signs, providing feedback, or allowing hand calibration. Non-functional requirements describe the overall qualities of the application, such as simplicity, performance, and data security.

For example, "the system must let users save progress" is a functional requirement, because it specifies a concrete feature. In contrast, "the system should be easy for beginners" is a non-functional requirement, because it relates to the general user experience rather than a specific action. Separate lists of functional and non-functional requirements are maintained and can be extended as new issues are discovered during development and testing.

3.3.2 Prioritising Features

With so many things to include, I saw I had to decide which parts were vital and which. Given the number of potential features, it was necessary to distinguish between essential elements and optional enhancements. The highest priority is given to reliable hand tracking, accurate recognition of ISL signs, basic real-time feedback for learners, and the ability to save and load user progress. Other aspects, such as more elaborate menus, extended quiz modes, or cloud synchronisation, are considered important but can be deferred until the core functionality is stable.

To help stay on track:

- Must-have: Core sign tracking, accurate feedback, calibration, basic navigation, and saving data.
- Should-have: More lesson modes, cloud sync, better accessibility, maybe progress charts.
- Nice-to-have: Fancy graphics, custom avatars, very detailed analytics, advanced quizzes.

This prioritisation helps keep the project manageable by focusing on delivering a simple, working application before adding more advanced features. The guiding principle is that a

stable, limited prototype is more valuable than an ambitious system that remains incomplete.

3.3.3 Making an Initial System Model

To clarify what needed to be built, an initial system model was created that describes how the main parts of the application interact.

The following modules were identified:

- **Hand Tracking Module:** This will oversee getting the data from Meta Quest 3's sensors and figuring out what gestures the user is making with their fingers and hands. Without this, nothing else in the app really works.
- **Feedback Module:** Every time a user tries a sign, this module compares what they did to what the sign should look like and then gives them a message or score ("Great job!" or "Try again"). This helps learners know how they're doing right away.
- **User Interface (UI):** The UI is supposed to be super clear, with menus that make sense and buttons that are big enough for VR. It also includes the tutorial, help tips, and any graphics or text that guide users during calibration and practice.
- **Data Storage:** I'll use JSON files and maybe later add cloud saving if I figure it out. It's not very fancy, but it keeps progress between sessions.

By modelling the system in modular form, each component can be developed, tested, and debugged independently, and upgraded later without disrupting the rest of the application.

3.3.4 Making an Initial System Model Use Case

A use case diagram was produced to describe how a typical learner interacts with Sign & Spell VR. The main actor in the diagram is the User, representing an individual practising ISL with the headset. The primary use cases include creating an account, logging in, calibrating hands, practising signs, completing tests, and viewing overall progress.

The diagram also shows that several higher-level use cases depend on shared internal functions. For example, Calibrate Hands, Practice Alphabet, Practice Words, and Test all include the Track Hand Movements and Recognise Hand Pose use cases, because the application must continuously capture and analyse hand data during these activities. Practice and Test also include Feedback and Save Progress, indicating that the system should provide real-time responses and update stored user data whenever a sign is attempted. This model helps to organise the main features and clarifies how user actions connect to the underlying tracking, feedback, and storage logic in Sign & Spell VR.

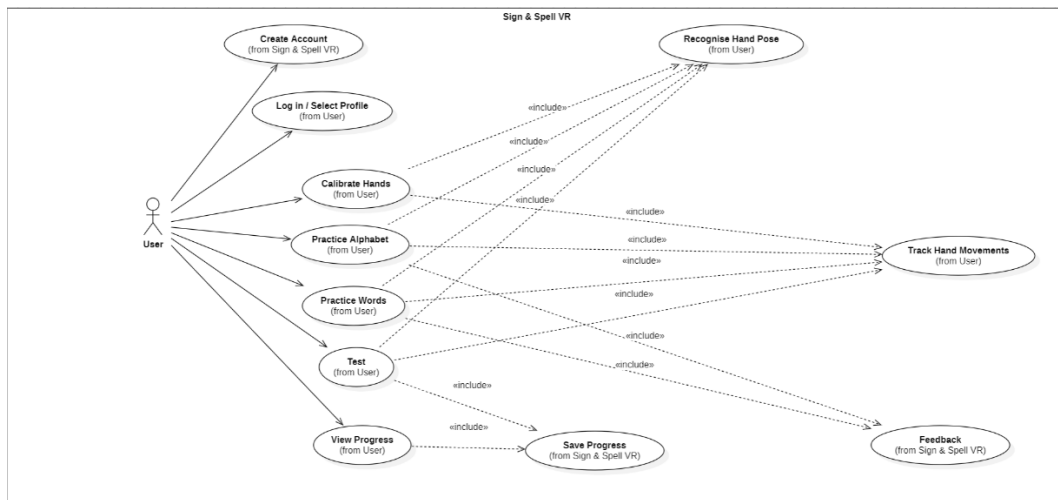


Figure: Use Case Diagram for Sign and Spell VR

3.3.5 Risk and Challenges

Several potential risks and challenges have been identified. A major technical challenge is achieving sufficiently accurate hand tracking; VR hand tracking can occasionally skip or misinterpret certain gestures, particularly ISL letters that are visually similar. Poor calibration may quickly lead to user frustration. Performance is another risk: if feedback logic or graphics become too demanding, the headset may lag or crash. There is also a usability risk that some users, especially those unfamiliar with VR, may find the controls confusing or intimidating.

To mitigate these risks, the project focuses on clear calibration procedures and simple user flows. Early user feedback, initially from a small group of testers, is planned as soon as a working prototype is available, so major issues can be identified and addressed before they become entrenched in the design.

3.3.6 Reviewing Gaps

Despite the planning carried out so far, there are still gaps that need to be addressed. Further investigation is required into how cloud synchronisation can be implemented in Godot and what security considerations apply when saving data online. Additional feedback from teachers and members of the Deaf community is also needed, since the current requirements are based largely on literature, existing applications, and the developer's perspective. It is expected that real users will reveal missing features or necessary adjustments once they begin to use the application.

3.4 Initial System Specification

This section brings together the key needs of the system into a structured specification that can guide development. It sets out what the application should do (functional requirements), the qualities it should exhibit (non-functional requirements), and an initial system architecture describing the main components and how they interact.

3.4.1 Initial Functional Requirements

These are the core functions the application must provide for users to be considered feature-complete:

- The system must track both hands and all five fingers using the Meta Quest 3 headset's sensors, so that users can practice ISL with proper finger shapes and positions.
- The app must let users calibrate the system for their own hands before starting, since different people might have different hand shapes or positions, and this can affect accuracy.
- The system should recognize and check ISL signs, giving the learner instant feedback on whether they signed correctly or need to try again.
- There needs to be a simple navigation system (menus for lessons, practice, calibration, and feedback) that is easy to use, especially for people new to VR.
- All user progress (like what signs have been learned, calibration data, and any scores or streaks) must be saved automatically, so users can pick up where they left off.
- Ideally, the app should include practice modes (like the alphabet, words, maybe even sentences later), and a way to track personal progress or see simple stats.
- If I get that far, the system can have an option to sync data online (cloud sync) so users aren't limited to one headset or device.

3.4.2 Initial Non-Functional Requirements

These requirements describe the overall feel, stability, and usability of the application, focusing on what the experience should be like rather than on specific features the user selects.

- The app should be really easy to start using, even for people with no VR or ISL experience. Menus and instructions need to be clear, big enough for VR, and not overloaded.
- It should run smoothly on Meta Quest 3; lag, crashes, or slow responses are likely to frustrate users and make them quit.
- The system should be reliable. Progress and calibration data shouldn't go missing, and the app should give clear error messages if something goes wrong.
- Accessibility should be good, so features like left-handed mode, and large, readable text are considered, making sure as many people as possible can use it.
- Data privacy is important especially if there is cloud saving, so user progress and settings should only be visible to them and not shared without permission.

- The app should be easy to maintain or update later. If I need to fix bugs or add new features in the future, it shouldn't require starting over from scratch.

3.4.3 Initial System Architecture

Having a plan for how the main parts of the system connect should make development easier later on. The following description outlines the architecture at this stage, with the understanding that it may change as the project progresses.

Main Components/Modules:

1. **Hand Tracking Module:**
 - Uses Meta Quest 3's built-in sensors and software (probably OpenXR) to track and get hand/finger position data in real time.
 - Feeds this data into the feedback engine.
2. **Feedback Engine:**
 - Compares user's hand positions to stored, "correct" ISL signs.
 - Gives feedback instantly in the app, like "Correct!" or shows what was wrong.
3. **User Interface (UI) Module:**
 - Handles all menus, calibration screens, practice lessons, and feedback pop-ups.
 - Designed in Godot 4 with big VR-friendly buttons and simple navigation.
4. **Data Storage Module:**
 - Stores user progress and settings using local JSON files.
 - Might connect to a cloud server/API for syncing later, depending on progress and what I can manage to learn in time.

3.4.4 How it all Works

- The user puts on the headset and chooses a practice mode from the UI.
- The Hand Tracking Module tracks their signs and sends info to the Feedback Engine.
- Feedback Engine checks if the sign matches the database of ISL gestures and returns feedback (correct/incorrect, hints).
- The UI displays response, score, or any helpful messages.
- Data Storage records new progress or calibration automatically.

3.5 Conclusions

Overall, this System Analysis section gives a clearer picture of what Sign & Spell VR is intended to be. By breaking the system down into users, key features, core modules, and likely risks, the project has a more structured direction to follow.

Much of the content is based on reviewing other sign language learning applications, reading relevant research for the proposal, and informally asking people what they would expect from a project like this. Separating the requirements into “must have” and “nice to have” items helps reduce the chance of getting distracted by extra features that are not essential. Focusing first on hand tracking, feedback, and ease of use should provide a practical system that is not overwhelming for beginners.

The planned architecture is relatively simple, but it offers a workable starting point and makes it easier to imagine how the different parts will fit together. It is expected that some elements will need to change as development continues and new issues appear.

There are still open questions, such as how to implement cloud synchronisation in practice, how best to handle hand-tracking errors, and how to make the application more accessible to a wider range of users. However, having an initial specification and a list of identified risks means that, if progress slows, there is at least a clear view of where the gaps are. Ongoing feedback from test users will be important to check that the system remains useful and respectful to its intended audience.

In summary, this section outlines a realistic plan that takes insights from the research and applies them to the system design. The next step is to begin implementing the core features, continue gathering feedback, and be prepared to adjust the design when parts of the system do not work as expected.

4. System Design

4.1 Introduction

4.1.1 Translating Requirements into Design

The purpose of this section is to describe how the requirements and specifications identified earlier will be transformed into a concrete design for the Sign & Spell VR application. This involves both planning how the software will be structured and deciding on the technologies to be used, with a clear focus on making the app functional, usable, and scalable. Effective design is important because it ensures that development proceeds smoothly, and that the final system meets user needs effectively.

4.1.2 Design Principles and Goals

In designing this system, the main principles guiding the process are simplicity, modularity, and user-friendliness. Simplicity helps ensure that users, especially beginners, can interact with the app without confusion. Modularity allows me as a developer to isolate and manage different parts of the system separately, making it easier to debug and add new features later. User-friendliness is vital to encourage learners to practice consistently, by providing clear feedback, easy navigation, and an immersive but not overwhelming VR environment. These goals reflect the requirements identified from prior research and user feedback.

4.2 Software Methodology

4.2.1 Incremental Model

For the development of the project, the Incremental Model has been chosen as the software development methodology. This process involves dividing the project into small parts or “increments,” delivering a functional system for each stage that progressively adds new features. The focus is on building and testing core features first, such as hand tracking, basic sign recognition, user feedback, and then adding more advanced features in later increments.

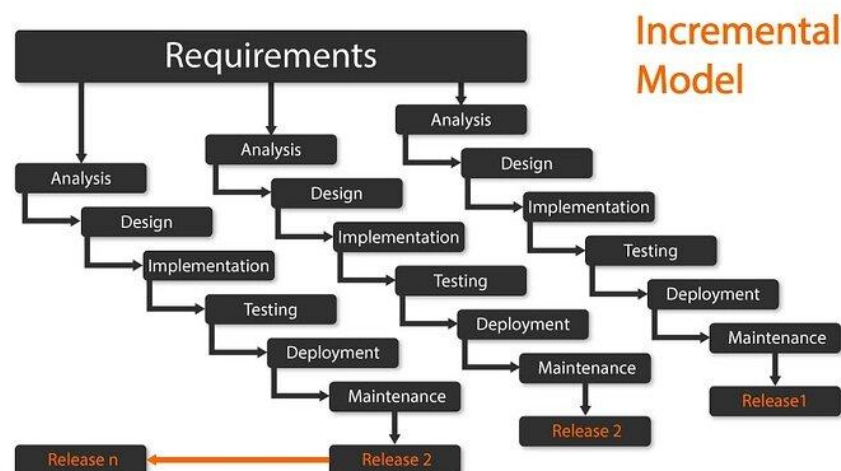


Figure 5: Incremental Model

4.2.2 Advantages of Incremental Development for my Project

This methodology offers several advantages for a solo project like mine. By developing the system in stages, I can continuously test and receive feedback on smaller parts, allowing me to identify and fix problems earlier. It also helps me stay motivated by achieving regular milestones and ensures that the most critical features are prioritized first. Additionally, incremental development supports flexibility, letting me adjust or reschedule features depending on progress and learning during development.

4.2.3 How Will I Apply it

In practice, I plan to start with a simple working prototype containing the core functionality: real-time hand tracking, recognition of basic ISL letters, and immediate feedback. This increment will focus on system stability and accurate tracking. Later increments will expand the practice set to include words and sentences, improve user interface polish, and implement features like data saving and cloud syncing. Each increment will be tested thoroughly before moving to the next.

4.3 Overview of System

4.3.1 Logical Architecture

The logical architecture is like a blueprint for the app. It shows how different parts of the system will work together to make the Sign & Spell VR app function smoothly. These parts are modules or components, each doing a specific job but communicating with each other, so the user experience is seamless. Having this clear makes it easier to build and test the app in pieces.

4.3.2 Main System Modules

Here are the key modules I plan to develop, along with what each does:

- **Hand Tracking Module:**
This is the foundation of the app. It captures the user's hand and finger movements via the Meta Quest 3's built-in sensors. It uses the OpenXR plugin in Godot 4 to get accurate tracking data. This module sends the raw hand pose data to the rest of the app continuously during use, allowing the system to "see" what the user is signing.
- **Feedback Engine:**
Once it receives the hand data, this module compares the user's gesture to the database of valid ISL signs. It decides if the sign is correct, partially correct, or needs more practice. It then sends feedback signals back to the UI, like a "correct" message or suggestions on how to improve. This module helps provide real-time responses, which is important to keep learners engaged.
- **User Interface (UI) Module:**
The UI module controls everything the user sees and interacts with inside VR. This includes menus for selecting practice modes, buttons for calibration, feedback pop-

ups after a sign attempt, and progress indicators. The UI is designed to be clear and simple, using larger elements suitable for VR so users aren't confused or overwhelmed.

- **Data Storage Module:**

This module manages saving and loading personal user data, such as calibration settings, user progress, and any practice statistics. It works with local JSON files initially, which is straightforward and well supported by Godot. Later versions may expand this to include cloud storage or syncing features, allowing users to access their data across multiple headsets or devices.

4.3.2.1 Data Model Class Diagram for User, Calibration and Progress

The class diagram below is a simple representation of the data the system needs, based on the requirements and ideas described in earlier chapters. The literature review and requirements analysis both track individual learners, storing calibration information, recording practice results and giving clear feedback, so the model focuses mainly on those parts for the VR application. The User class holds basic details about each learner and links to Calibration, which keeps the per-user pose data needed for the hand-tracking and recognition features discussed in Chapter 3.

The Progress class stores scores and status for each practice module, which matches the way progress and motivation are mentioned in the research and in the incremental development plan. By connecting Progress to Feedback, the diagram also shows how short feedback messages for specific signs or attempts can be saved, supporting the real-time feedback loop described in the data-flow section. Finally, the DataStorage class is included to reflect the practical design choice of using JSON files and Godot's FileAccess API instead of a full database; it is not very sophisticated, but it is enough for the current prototype.

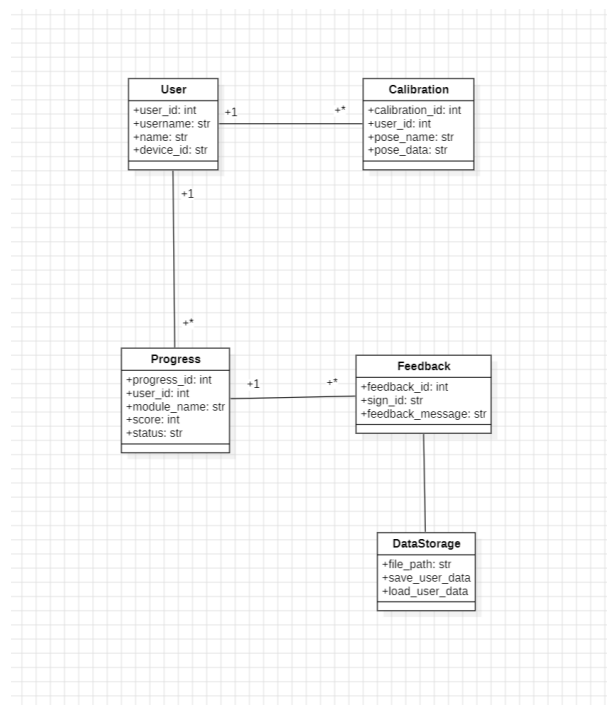


Figure: Class Diagram for Sign and Spell VR

4.3.3 Physical Architecture and Development

The entire system is designed to run directly on the Meta Quest 3 headset, which is a standalone virtual reality device. This means that all the processing required for the app, including tracking hands, giving feedback, displaying menus, and saving data, will happen within the headset itself rather than relying on an external PC or server. This makes the system more portable and easier for users to set up, because they don't need extra hardware or complicated connections.

Since everything runs on the device, users can just put on the headset and start practicing right away, without needing to be connected to a computer. Development of the app will be done using Godot 4, which supports OpenXR. This is the technology used by the Quest for VR applications. Godot makes it possible to build VR apps that work smoothly on the Quest without needing extra software.

All app files, user profile data, and temporary cache will be stored on the internal storage of the headset. This ensures fast access and reduces the chances of data loss. If I add future features like cloud syncing, those options will connect to online servers, but the core system will operate independently offline.

The goal is to deliver a user-friendly, wireless experience that is reliable and straightforward for anyone to use, whether they're teachers, students, or just beginners exploring sign language in VR. The system setup is intended to be simple and efficient, making it suitable for different environments like classrooms, homes, or labs.

4.3.4 Data Flow Between Modules

The system's data flow describes how information moves between the different parts of the system when a user interacts with the app. Here's how a typical session would go, step by step:

- **User Interaction:**
When the user puts on the VR headset and opens the app, they select a lesson mode or practice activity from the main menu.
- **Hand Tracking Module:**
This module is always active while practicing. It continuously collects data about the user's hand and finger positions using the sensors on Meta Quest 3. The module sends this positional data in real time to the Feedback Engine.
- **Feedback Processing:**
The Feedback Engine receives the hand data and compares it against a stored database of correct ISL signs or gestures. It checks whether the user's signs match the correct signs closely enough to be considered correct. It then generates feedback, such as a visual message ("Good job!") or hints if the sign was wrong.
- **User Feedback Interface:**
The feedback results are immediately sent to the UI module, which displays

messages, score indicators, or visual cues directly in VR. This helps the user immediately understand their performance and adjust.

- **Progress Storage:**

After the practice session, the system automatically saves the user's progress, including calibration settings, completed lessons, and scores. This data is stored locally in JSON files but could later be synced online if I add cloud features.

This data flow ensures that the system remains responsive, engaging, and easy to use. All modules coordinate smoothly to provide real-time feedback and keep track of the user's improvement.

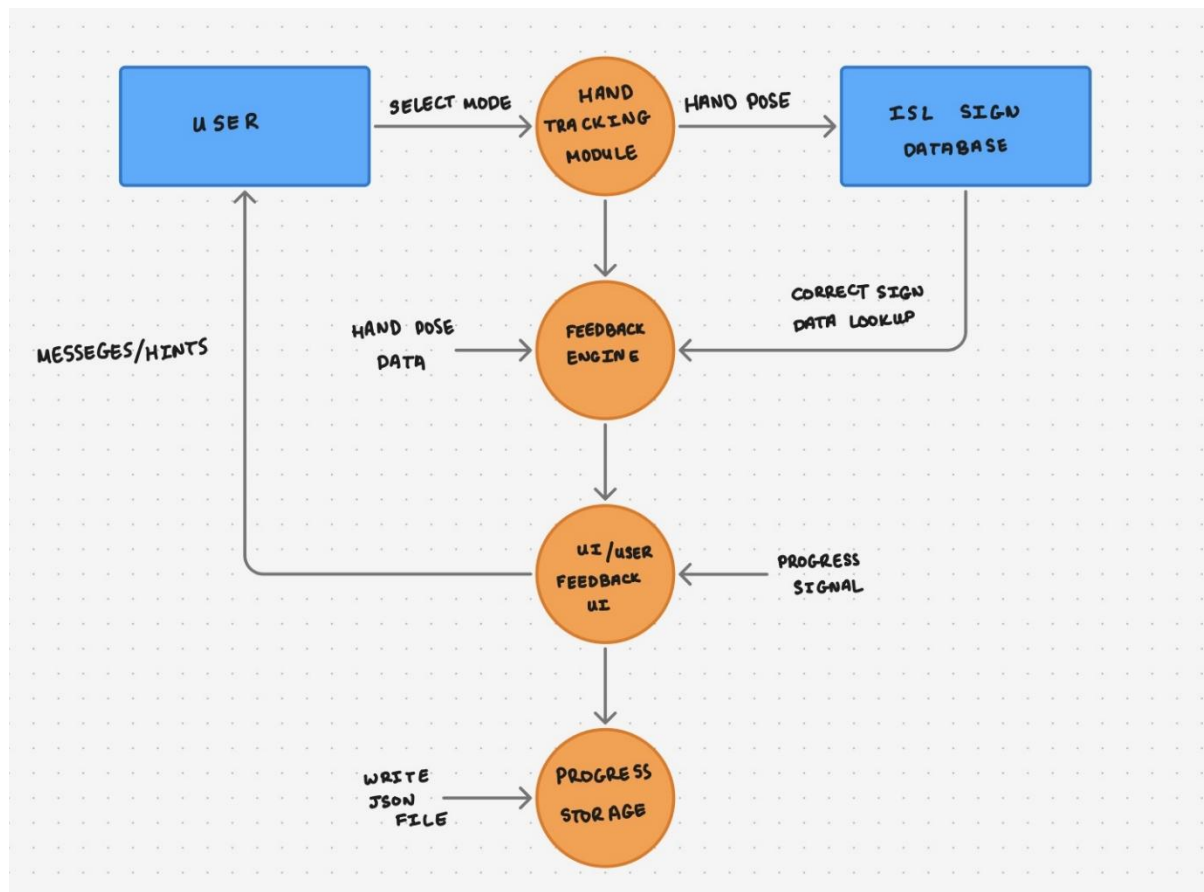


Figure 6: Data Flow Diagram Between Modules in Sign & Spell VR

4.3.4 Modular Benefits

Using a modular system design means I organize the app into different independent parts or modules, such as hand tracking, feedback, UI, and data storage. This approach offers many benefits for development:

- **Easier Testing:**

I can develop and test each module separately. For example, I can perfect the hand tracking module without worrying about the feedback system or UI. If something doesn't work, it's easier to pinpoint and fix the problem.

- **Simpler Debugging:**
When bugs happen, I know exactly which module is causing the problem, making it faster to troubleshoot and fix them.
- **Future Expansion:**
If I want to add new features later, such as cloud syncing, progress charts, or more complex feedback, I can develop those parts independently and plug them into the existing system without rewriting everything.
- **Better Maintenance:**
The system is less likely to break when I update or change parts of it because each module is separate. This makes it easier to keep the app running smoothly over time.

Overall, the modular design makes development more manageable, especially for a solo project like mine. It provides a clear structure, reduces complexity, and helps me stay organized as I build and improve the app.

4.4 Design System

4.4.1 Applying Design Approach

For the design of the Sign & Spell VR system, I have used a structured design methodology to translate the specifications and requirements into a detailed, workable plan. The goal of the design phase is to create clear models and diagrams that describe how each part of the system will function and interact, providing a roadmap for development. This methodology focuses on modularity, clarity, and scalability, which aligns well with the incremental development approach I am following.

4.4.2 Modular Design Approach

The system design follows a modular approach, dividing the application into distinct components, each responsible for specific tasks. This allows each module to be developed and tested independently before integration. The main modules, as identified earlier, include:

- **Hand Tracking Module:** Designed to interface smoothly with the hardware sensors, this module will interpret the raw data into user hand poses.
- **Feedback Module:** Responsible for interpreting hand poses and determining sign correctness, this module will use algorithmic matching techniques based on a sign database.
- **User Interface Module:** This module will manage all user interactions, including menus, practice sessions, feedback displays, and calibration workflows, all designed for clear VR visualization.
- **Data Storage Module:** Tasked with saving and loading user profiles, progress data, and calibration settings, this module will use lightweight JSON files for local storage with future cloud integration planned.

By designing these modules separately but ensuring clear interaction points, the system stays maintainable and flexible for future feature upgrades.

4.4.3 Use of UML Diagrams

In this project, UML diagrams are used in a simple way to support the design rather than as a full modelling approach. A use case diagram in Chapter 3 is used to summarise how learners interact with the main features of the Sign & Spell VR system, and a class diagram in Section 4.3.2.1 models the core data entities such as User, Calibration, Progress and Feedback. These diagrams provide a basic visual view of the system and were mainly helpful for clarifying the data model and key interactions before and during implementation.

4.4.4 Prototyping and Iteration

The design methodology emphasizes early prototyping like building simple versions of each module to test concepts and gather feedback. For example, an initial prototype of the hand tracking will confirm how accurately the Meta Quest 3 sensors can detect finger positions in Godot 4. Similarly, a basic prototype of the user interface will help verify menu navigation and feedback readability in VR.

This iterative process allows me to revise the design continually based on testing results and user feedback, leading to a more robust and user-friendly final application.

4.4.5 Scalability and Flexibility Considerations

During the design, I have taken care to keep the system scalable and flexible. For example:

- The feedback engine is designed so that adding new signs or improving the recognition algorithm won't require a complete rewrite.
- The data storage module can be extended to support cloud syncing or networked data without affecting other parts of the app.
- The modular UI allows for easy changes to the layout or addition of new practice modes.

By keeping these principles in mind during design, I aim to build a system that not only meets the current requirements but can also accommodate future enhancements.

4.5 Conclusions

In this chapter, I have outlined how I plan to design the Sign & Spell VR system by translating the earlier requirements into a clear and manageable software design. By choosing the incremental development model, I can focus on delivering functional parts of the system step-by-step, allowing me to build core features first and add enhancements later. This method supports testing, feedback, and improvement at every stage, which I believe is ideal for this project.

My design adopts a modular approach, dividing the system into well-defined components such as hand tracking, feedback, user interface, and data storage. This separation makes the development process easier to handle and helps me focus on one part at a time, which is especially useful for a solo project. The modular design also provides flexibility for future

upgrades, whether that's adding cloud syncing, more lesson types, or better accessibility features.

I have planned to use standard design tools like UML diagrams to visualize the system and its workflows before diving deeply into coding. This will help ensure I have a solid foundation and reduce bugs or mistakes. Additionally, early prototyping and iteration are important aspects of the design process, paving the way for a user-friendly and robust application.

Although there are challenges ahead, such as ensuring accurate hand tracking and smooth user interaction in VR, the structured design and incremental approach give me confidence that these issues can be addressed step-by-step. Overall, this design chapter lays a strong groundwork for the next phase of development, ensuring the system will be effective, expandable, and enjoyable to use.

5. Testing and Evaluation

5.1 Introduction

In this section, I will explain the testing and evaluation methods I plan to use for the system. Since I am still working on the project, I have focused on deciding how to test the parts I have made so far and how to check if they work well. Testing is important to make sure each part of the system does what it is supposed to do. Evaluation is also important to see if the system is easy to use and useful for learning.

5.2 Plan for Testing

The system has different parts like hand tracking, giving feedback, user interface and saving data. Each part needs different ways to test it:

- For the Hand Tracking Module, I will test how well the system recognizes the signs of the Irish Sign Language alphabet and some words. This will involve me signing different letters first and then words to see if the system detects them correctly. I will also try to test the hand tracking in different lighting conditions and angles to see if it still works well, because lighting can sometimes affect VR sensors. I will also ask some friends to test this part so I can get more varied feedback.
- To get more input, I am giving my friends my VR headset to try out the system themselves. They will use and test the main features like signing letters and sentences and then tell me what they think about how well it works. They will also let me know if any parts are confusing or if the system fails to recognize their signs. Their feedback is important because they might find different problems than I do, and they might have ideas for improvements or new features.

- The Feedback Module needs to be tested by checking if the messages and hints shown to users are correct. For example, when the system detects a correct sign, it should say so clearly, and if it's the wrong sign, it should give hints or corrections. I will test this by signing right and wrong signs intentionally and observe if the feedback matches what I did. This part will be tested mostly manually, but I plan to write some small test cases in the prototype to double-check basic things automatically.
- Testing the User Interface (UI) means going through all the menus, buttons, and prompts to make sure they work how they should. I will test if buttons respond correctly when clicked and if the text instructions are easy to understand. I will also check if users can navigate between different parts of the application smoothly without getting lost or stuck. Again, I will ask my friends to try the UI as well and report any parts that feel confusing or awkward in VR.
- For the Data Storage Module, I will try saving my user settings, hand calibration data, and progress, then closing and reopening the application to check if the data loads back correctly. This will make sure users do not lose their settings or progress when they stop using the system. I will create multiple test profiles and check if data keeps correct values for each profile.

Testing will be done continuously and bit by bit while I keep working on the system. I plan to test every new feature, instead of waiting until the end. This way, I can find mistakes early and fix them before moving on to the next part. I think this approach helps prevent big problems from building up and makes it easier to improve the system step by step.

5.3 Plan for Evaluation

Before developing the system, I had my friends try existing Irish Sign Language learning apps and ASL apps to understand their features and limitations. Their feedback helped me identify requirements and informed the design of the Sign & Spell VR system. This early comparison also guides what aspects I should focus on when testing and evaluating my own system.

In this section, I will describe how I plan to check if each part of the system meets its goals and how I will understand what works well or what needs improvement. This includes different methods used for different components to get useful feedback.

- For the Hand Tracking Module, I will evaluate how accurate and responsive the system is when recognizing ISL signs. This will be done by having people use the system to sign letters and simple words, and I will observe whether the system correctly detects what they sign. The users will also give me their opinions on how well it feels, for example if it is smooth, fast, or delayed. This helps find areas that might need fixing to improve recognition.
- The Feedback Module will be evaluated by checking if the feedback messages or hints given to users are clear and helpful. Users will tell me if the feedback helps them correct their signs or if the messages are confusing. This evaluation focuses on matching the feedback with what the user expects to get.

- The User Interface evaluation will focus on how easy it is for users to navigate and use the features in the VR environment. I will watch users as they complete tasks and look for any confusion or errors. Then, I will ask them simple questions about whether they found the interface friendly and if instructions were clear. This kind of evaluation is important to make the system more user-friendly.
- For the Data Storage Module, evaluation will ensure that user data is saved and loaded correctly. I will simulate typical user behaviour, saving progress and settings, quitting, and then restarting the system to confirm data persistence. If there are problems with data not being saved or lost, this part of the system will need improvement.
- When evaluating the Overall System, I will check if users find it helpful and motivating for learning Irish Sign Language. I plan to ask users before and after they try the system about their confidence in signing. This can help measure if the system supports learning. Technical issues such as crashes or delays will also be monitored because these can affect user experience.

At this stage, the evaluation will be mostly informal and involve a small number of users, mainly friends testing the prototype. Their feedback will guide changes and improvements for the next phase of the project. In the final report, more formal evaluation with a larger user group and structured tests will be planned to better measure outcomes and usability.

5.4 Conclusions

Overall, the testing and evaluation plans I have set up so far give me a good way to check how well my system is working at this stage. Since the project is not finished yet, the testing done until now has been mostly informal and focused on core parts like hand tracking and feedback from existing apps and prototype. Having my friends try out my system, existing apps, and give feedback has been helpful to find problems I didn't notice and get ideas for making it better.

In conclusion, this plan for testing and evaluation is a good starting point for making sure the Sign & Spell VR system will work well and meet users' needs. It also shows that I am thinking about how to improve the system based on real feedback. I will keep testing new features and asking users for their opinions so I can make the best possible VR learning tool for Irish Sign Language.

6. System Prototype

6.1 Introduction

For my prototype, I mainly wanted to see if I could get hand tracking and basic sign detection working in VR. I used Godot 4 because that's what we're learning in class, and it also lets me use the Meta Quest 3 headset to try out hand tracking. I didn't have much experience with VR projects before, so I tried to keep everything simple and focused on learning the basics first.

I started by adding a sign language alphabet chart to the environment so I could easily check what each sign should look like. This is the chart I found online that represents the Irish Sign Language.

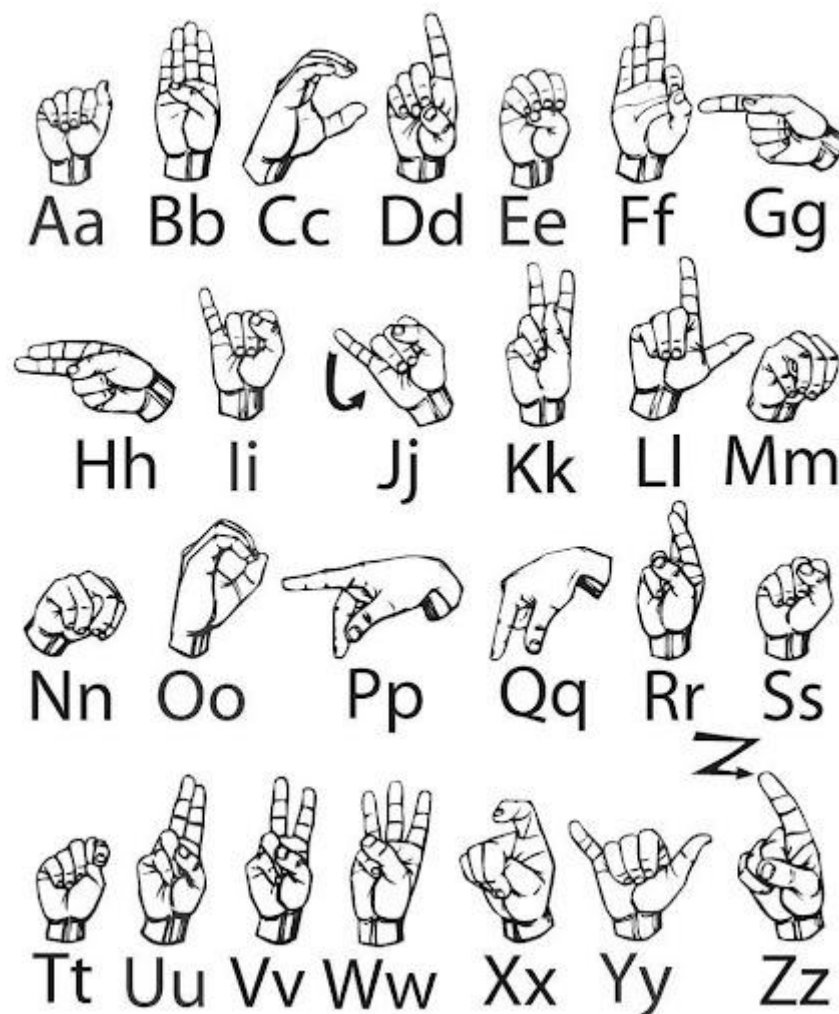


Figure 7: ISL Alphabet

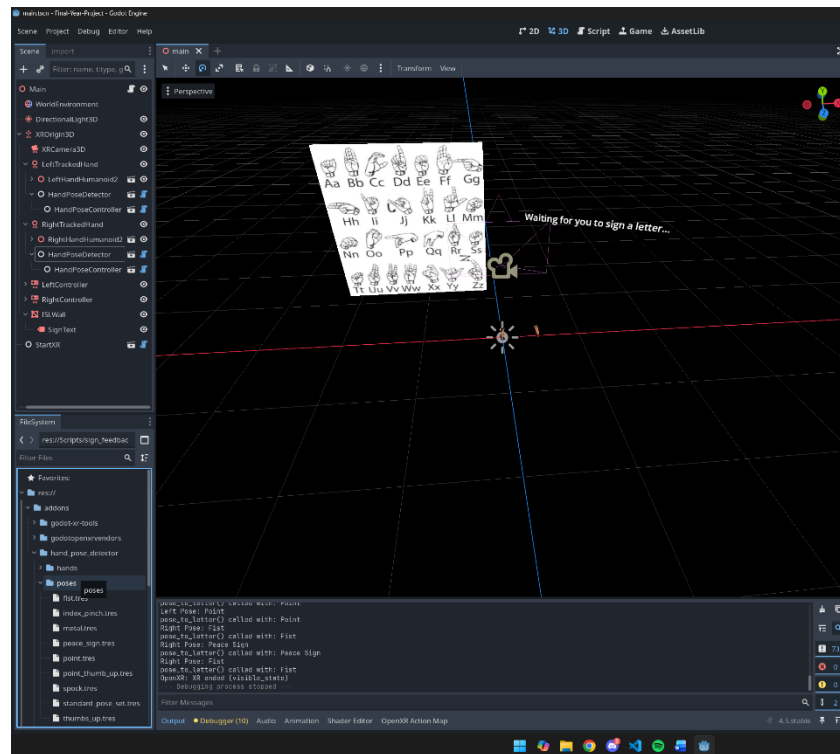


Figure 8: Current Environment for Prototype

I installed some Godot addons, like the OpenXR plugin and the hand pose detector, to help with tracking the hands and recognizing sign shapes. Most of my time went into writing scripts that try to match the hand poses to ISL letters and show feedback in the VR scene.

Currently, the code just handles a few signs, since some VR hand poses can look similar and I wanted to test if things worked before trying to cover every letter. My screenshots and code examples show how far I got. There's still a lot to improve, but I managed to get the basics running for this first version.

6.2 Prototype Development

For the prototype, I split my project into a few simple modules so I could work on each part without getting lost. Godot's OpenXR addon helped a lot, because it already includes built-in hand tracking and poses detection features. That saved me from having to write everything myself and let me focus on connecting the blocks in my app.

6.2.1 Hand Pose Detector Setup

I started by setting up hand tracking for both hands in the scene using the OpenXR plugin and its helper nodes. The plugin provides the basic controllers and detectors I needed, so I just referenced them in my script and connected the signals.

```

extends Node

# Get References to Hand Pose Detectors
@onready var left_detector = $XROrigin3D/LeftTrackedHand/HandPoseDetector
@onready var right_detector = $XROrigin3D/RightTrackedHand/HandPoseDetector

# Reference to Label3D (SignText)
@onready var sign_text = $XROrigin3D/ISLWall/SignText

var left_letter := ""
var right_letter := ""

func _ready() -> void:
    >I # Connect signals from both detectors
    >I left_detector.pose_started.connect(_on_left_pose_detected)
    >I right_detector.pose_started.connect(_on_right_pose_detected)
    >I

```

Figure 8: Code Snippet for Hand Pose Detector

This means the app can automatically listen for hand pose changes as soon as it starts. I didn't have to write the detector code from scratch, the OpenXR addon does it all behind the scenes.

6.2.2 Signal Handlers for Pose Detection

Whenever the user makes a new hand shape, the detectors fire signals that call these functions. This is how the system figures out which pose was made and gets ready to update the UI.

```

# Signal Handlers
func _on_left_pose_detected(pose_name: String) -> void:
    >I # Debug Message:
    >I print("Left Pose: ", pose_name)
    >I left_letter = pose_to_letter(pose_name)
    >I update_display()
    >I

func _on_right_pose_detected(pose_name: String) -> void:
    >I # Debug Message:
    >I print("Right Pose: ", pose_name)
    >I right_letter = pose_to_letter(pose_name)
    >I update_display()

```

Figure 9: Code Snippet for Signal Handling for Pose Detection

These handlers keep things simple. They react whenever a new pose is made and move on to checking which letter it might be. The debug messages help me show what pose I am doing on the terminal.

6.2.3 Mapping Hand Poses to ISL Letters

After a pose is detected, my code tries to match it to an ISL letter. Right now I only mapped three poses (fist, point, peace sign) to letters, since they were easiest to set up and test and some VR hand poses look really similar.

```
# Pose -> Letter Mapping
func pose_to_letter(pose_name: String) -> String:
>| # Debug message
>| print("pose_to_letter() called with: ", pose_name)
>| match pose_name:
>| >| "Fist":
>| >| >| return "A"
>| >| "Peace Sign":
>| >| >| return "V"
>| >| "Point":
>| >| >| return "D"
>| >| _:
>| >| >| return ""
>| >| >|
```

Figure 10: Code Snippet for Mapping Hand Poses to ISL Letters

This function is basic but does the job for now. I plan to expand it later so the app can recognise more signs and letters, and find a way to get all the letters to be not in the OpenXR Addon, I would have to make it myself.

6.2.4 Updating UI Feedback

Once the system knows what pose was made, it needs to let the user know. This small function updates the sign text label in VR, so the user can see in real time which letter was detected for each hand.

```
# Update Text on Label
func update_display() -> void:
>| sign_text.text = "Left hand = %s\nRight hand = %s" % [
>| >| left_letter,
>| >| right_letter
>| ]
```

Figure 11: Code Snippet for Updating UI Feedback

Although it is not fancy, it works and lets the person practicing know if their poses match what the system expects.

6.2.5 Using Godot Addons, Environment Setup, and Hierarchy

To keep my project organized, I set up the scene using Godot's node hierarchy. The root node is called Main, which holds everything else in the environment.

Under Main, I have the WorldEnvironment and DirectionalLight3D nodes for lighting and general world setup. The XROrigin3D node is the starting point for setting up VR tracking. Inside XROrigin3D, there are nodes for the VR camera (XRCamera3D), and two main tracked hand nodes: LeftTrackedHand and RightTrackedHand.

For each tracked hand, I added human hand models (LeftHandHumanoid, RightHandHumanoid), plus the important HandPoseDetector and HandPoseController nodes. These detectors and controllers handle most of the hand pose tracking and processing, the OpenXR addon already includes the code to do this automatically, so I just needed to include these nodes in my scene.

The right hand branch also has extra nodes for meshes and skeleton (Skeleton3D, RightHandHumanoidMesh, XRHandModifier3D), which help make the hand tracking more accurate and realistic.

In addition to the hands, there are simple controller nodes (LeftController and RightController), which I might use later for extra inputs.

Next, there's the ISLWall node. On this wall, I placed my sign language alphabet chart for easy reference. Attached to ISLWall is the SignText label node, which updates in real time to show the letter feedback for each hand.

Finally, StartXR is at the bottom, which handles starting up XR/VR mode in Godot.

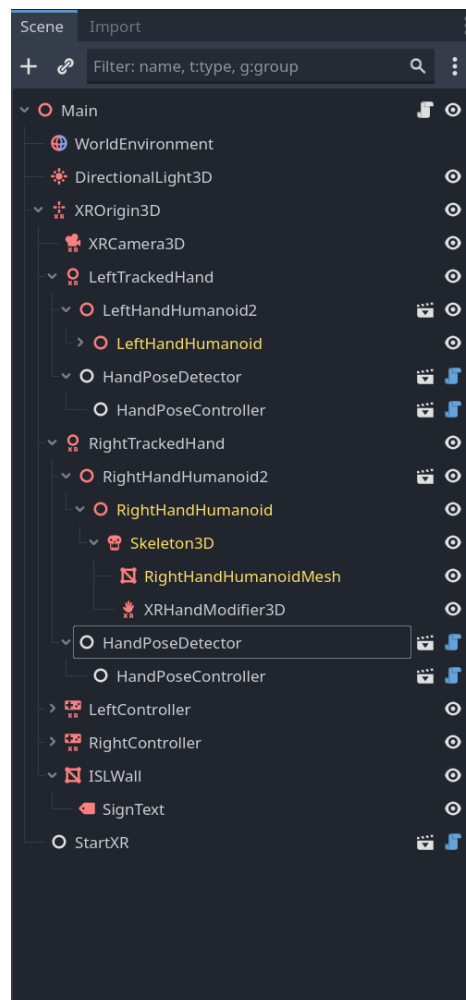


Figure 12: Current Hierarchy for Prototype

6.3 Results

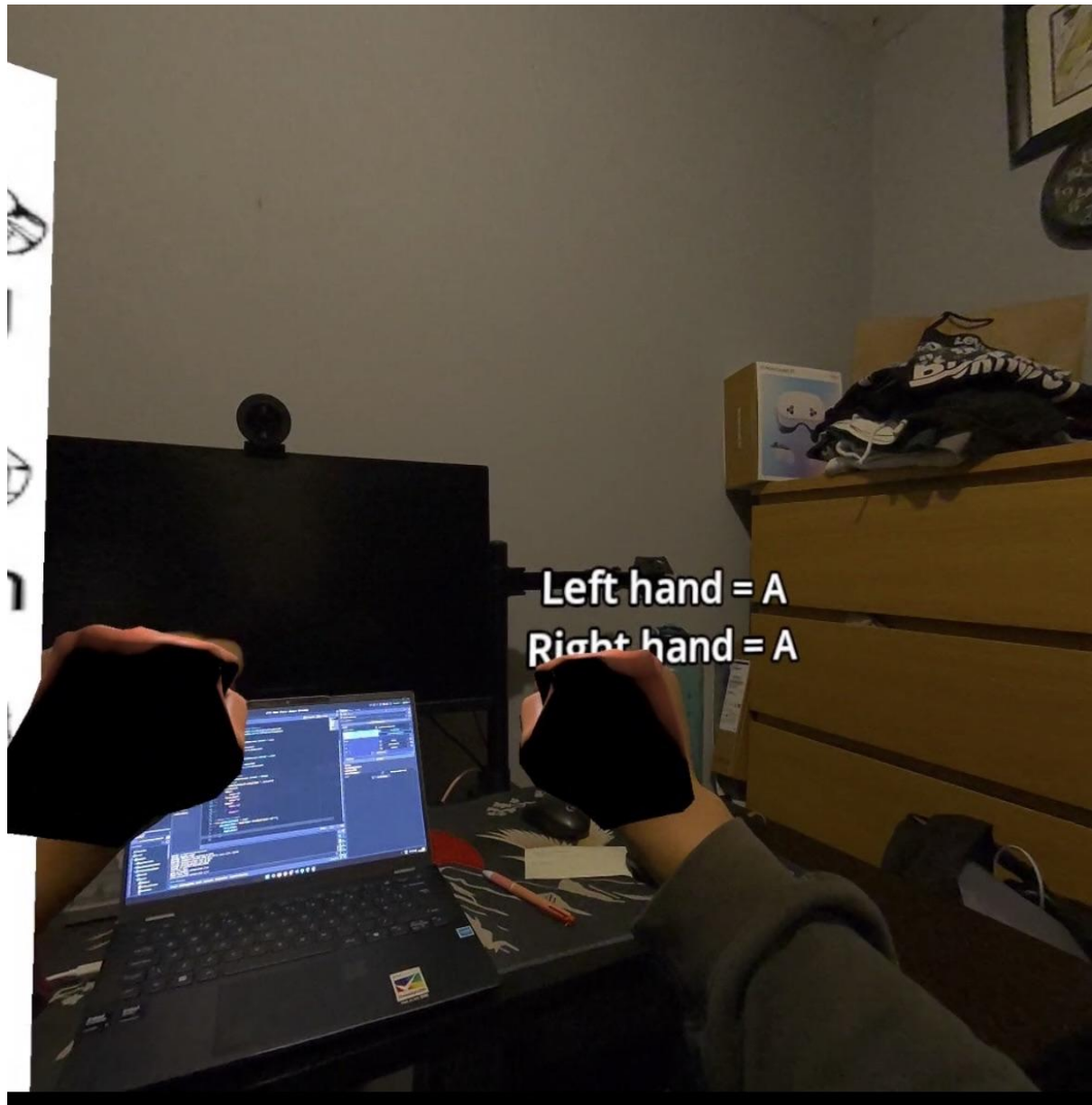


Figure 13: Test 1

This is my first test. On the ISL Alphabet, a fist is represented as “A”. It is successful as I both had my hands as a fist and the feedback says that I am signing “A”.

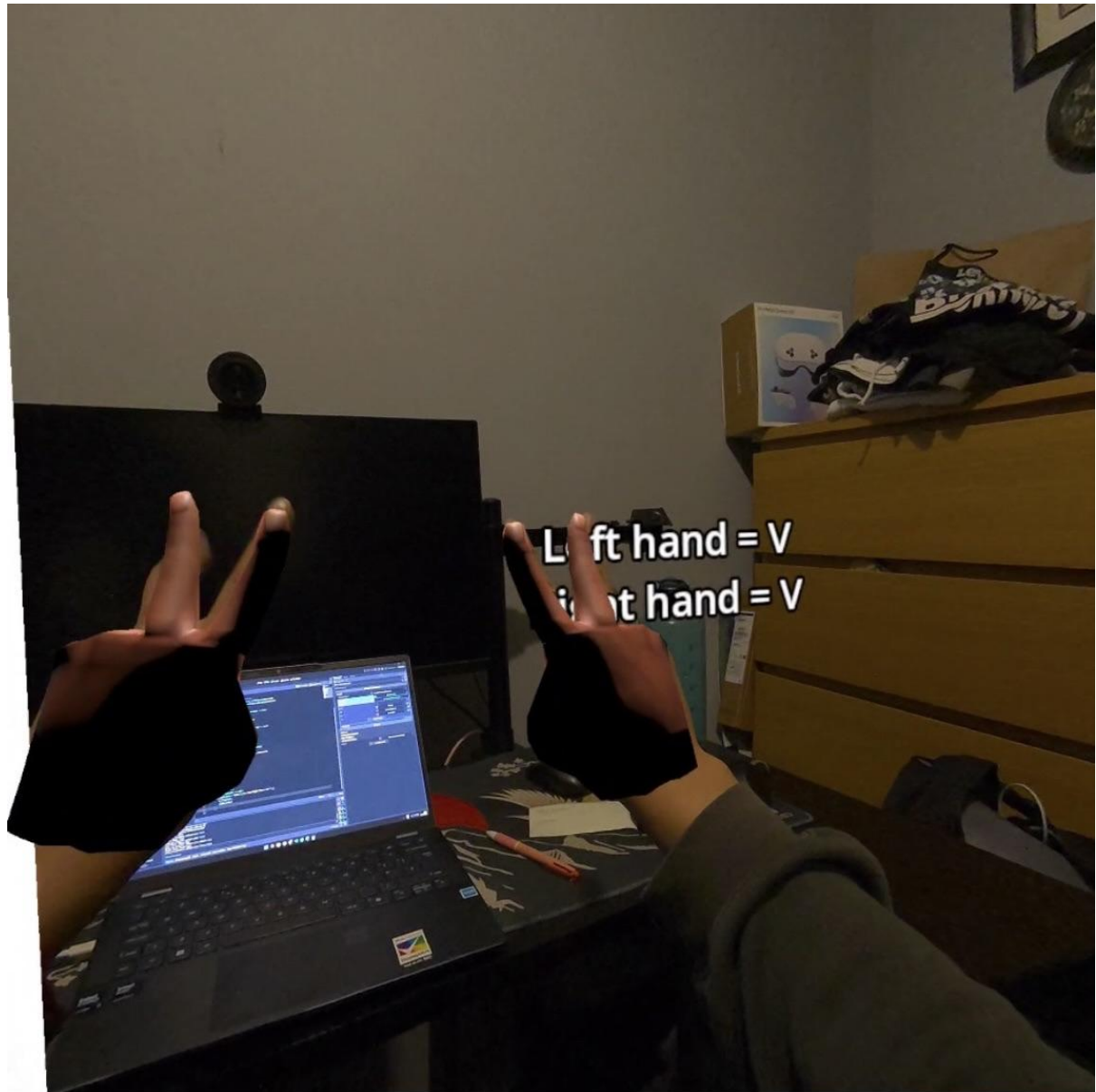


Figure 14: Test 2

This is my second test. Again, on the ISL Alphabet, posing a Piece Sign represents as "V". I both posed with both my hands and results were successful again. Both result in "V".

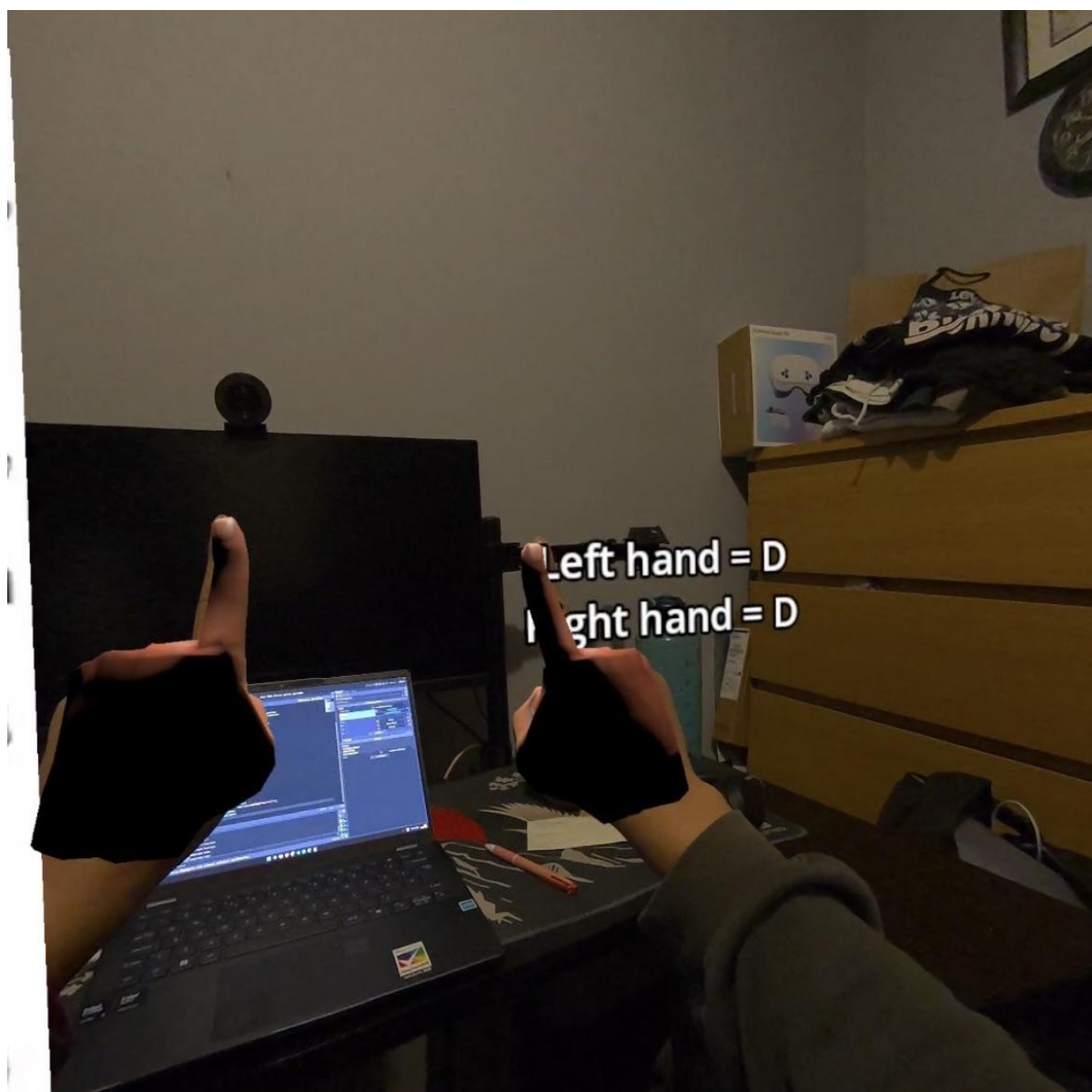


Figure 15: Test 3

This is my third test. On the ISL Alphabet, posing a Point represents as "D". I both posed with both my hands and results were successful again. Both result in "D".



Figure 16: Test 4

This is my fourth test. I tried posing different signs on both my hand. In the image, my left hand is in a Fist which represents "A" and my right hand is a Piece Sign representing "V". The outcome is successful.

6.4 Evaluation

To evaluate my Sign Spell VR prototype, I used the testing and evaluation approaches from Section 5, focusing mainly on functional tests of hand tracking, pose recognition, feedback, and the user interface.

6.4.1 Hand Tracking Accuracy

I tested the hand tracking module by signing different ISL letters (using fist, point, and peace sign shapes) while wearing the Meta Quest 3 headset. The system was generally able to

detect these core poses and print the correct letters to the screen. Sometimes, the hand pose detector would mix up similar signs (especially when finger positions weren't clear enough), but overall it worked well for basic shape recognition.

6.4.2 Feedback and UI

The feedback module successfully updated the VR sign label in real time whenever my hand changed poses. This helped me check if the system was responding instantly, which is important for a good learning experience. The alphabet chart in the VR space also made it easier to compare my hand shapes with the correct signs.

6.4.3 User Testing

I asked a few friends to try out my system, and their feedback helped me see what needs improvement. While most found the interface straightforward and easy to use once they got used to the basic gestures, they pointed out a couple of important limitations. Firstly, the app currently only supports recognition for a few letters, which made it seem limited, but they suggested I should try cover the whole ISL alphabet in future versions, so the app is more useful.

Testers also noticed that the hand pose detection can depend a lot on lighting conditions in the room. Poor lighting or shadows made it harder for the system to consistently pick up accurate finger positions, resulting in some missed or incorrect letter detections.

On the positive side, everyone agreed the UI was simple and not distracting, which made it easier to focus on signing rather than figuring out controls. Their comments helped me spot bugs and understand which features to work on next.

6.4.4 Plans for Further Testing

Based on what I learned from user testing, I plan to reflect on their feedback and keep working on the main features that matter most. For my next version, I will write more automated test cases and try using the system with a wider group of users, so it's not just my friends testing it. The feedback highlighted that people want all ISL letters to work, so I aim to expand the pose recognition to cover the full alphabet and improve calibration for better accuracy. I also know now that lighting can affect hand tracking, so I'm going to try testing in different environments to make sure the system works reliably.

6.5 Conclusions

To sum up, I've managed to build a basic VR sign language system that can track hands and recognise a few signs using Godot and the Meta Quest 3. The prototype does the main things I wanted detecting hand shapes and showing feedback on the letters, but it's still limited because it doesn't cover all the ISL alphabet yet and sometimes gets confused depending on lighting. User testing and my own experiments showed the system works okay for what it is, but there's a lot I still must improve. I plan to keep fixing things, add more features, and make the app better step by step as I learn more and get more feedback.

7. Issues and Future Work

7.1 Introduction

This section explains some of the problems or unexpected results encountered during the project so far. It also identifies risks that could affect finishing the project and outlines plans for continuing the work in the future, especially for next semester.

7.2 Issues and Risks

During development, some things worked, but a lot didn't go as planned. The hand tracking using OpenXR is still a bit off sometimes but it doesn't pick up certain gestures, and lighting in the room really affects how well it detects the hands. Because the prototype is only about halfway done, I haven't built out all the main features yet, so I couldn't do a full evaluation or lots of user tests.

Trying to juggle this project with other college work and assignments has also been hard. Time management was a big issue, and I ended up running out of time for some features I wanted to try. Another risk is that I only asked a few friends to test it, so I still don't know if the app will work for everyone, especially for people new to VR or sign language. There's also a chance that, as I add more modules (like saving data), some new bugs or crashes could happen and slow things down even more.

7.3 Plans and Future Work

Next semester, the main plan is to finish all the core features of Sign & Spell VR and make it more stable and usable. This includes improving hand tracking so it's more accurate, make feedback system, good UI for the users so they don't get confused, and a way to implement Data Storage.

To deal with my time management issue, I plan to organise the work into smaller tasks and spread them across the weeks of the semester. Setting clearer weekly goals should make it easier to keep track of progress and avoid leaving too much work until the end.

For evaluation and testing, I want to get more users involved, not just friends, so I can collect better feedback on how easy the system is to use and how helpful it is for learning Irish Sign Language. This might include short testing sessions with classmates or volunteers who are interested in ISL or VR. Their comments will help guide changes to the user interface and overall experience.

Fixing integration issues will also be an important part of the future work. Once more features are implemented, I will need to test how everything works together in the VR environment and look for crashes, delays, or unexpected behaviour.

7.3.1 Project Plan with GANTT Chart



Figure 17: Gantt Chart for Next Semester Work

Based on my future plans, I will complete the project by focusing on finishing the core features first, like hand tracking, feedback, user interface, and data storage. I will develop each part step by step and test it right after to make sure it works before moving on. After these modules are working well separately, I will combine them into the full system and do more testing with users to find any problems.

To keep things realistic, I will break down my work into smaller tasks and spread them out over the weeks from now until the final deadline. I will set weekly goals, for example finishing the hand tracking module by mid-December, then its testing by the end of December, and so on for the other parts. This way I can track my progress better and avoid rushing everything at the end. Regular user testing and fixing bugs will be a big part of the schedule too.

Finally, I will leave enough time to polish the system and write the final report properly. This plan should help me finish the project on time and make sure it works as intended.

References

- [1] ASL Fingerspeller – SideQuest. Available: <https://sidequestvr.com/app/1317/aslfingerspeller>
- [2] Silent Classroom VR – Meta. Available: <https://www.meta.com/experiences/silentclassroom-asl-vr/9249215595132131/>
- [3] ASL Champ! – ScienceDirect. Available: <https://www.sciencedirect.com/science/article/pii/S2949678024000096?>
- [4] DEF-ISL – App Store. Available: <https://apps.apple.com/mt/app/def-isl/id6447841373>
- [5] D. Novaliendry et al., “Design of Sign Language Learning Media Based on Virtual Reality,” *Int. J. Online Biomedical Engineering (iJOE)*, vol. 19, no. 16, pp. 111–126, 2023.
https://www.researchgate.net/publication/375642351_Design_of_Sign_Language_Learning_Media_Based_on_Virtual_Reality
- [6] VR Sign Language Learning with Hand Tracking [Video]. YouTube, 2023. Available: <https://youtu.be/HeFut3Htrcw>
- [7] “Getting Started with Godot: The Open-Source Game Engine,” *Meta.com*, 2022.
<https://developers.meta.com/horizon/blog/getting-started-with-the-open-source-game-engine-godot/>
- [8] “XR,” *Godot Engine documentation*, 2025. <https://docs.godotengine.org/en/4.4/tutorials/xr/index.html>
- [9] Danielle Bragg, Naomi Caselli, Julie A. Hochgesang, Matt Huenerfauth, Leah Katz-Hernandez, Oscar Koller, Raja Kushalnagar, Christian Vogler, and Richard E. Ladner. 2021. The FATE Landscape of Sign Language AI Datasets: An Interdisciplinary Perspective. *ACM Trans. Access. Comput.* 14, 2, Article 7 (June 2021), 45 pages. <https://doi.org/10.1145/3436996>
- [10] Danielle Bragg, Oscar Koller, Mary Bellard, Larwan Berke, Patrick Boudreault, Annelies Braffort, Naomi Caselli, Matt Huenerfauth, Hernisa Kacorri, Tessa Verhoef, Christian Vogler, and Meredith Ringel Morris. 2019. Sign Language Recognition, Generation, and Translation: An Interdisciplinary Perspective. In *Proceedings of the 21st International ACM SIGACCESS Conference on Computers and Accessibility (ASSETS '19)*. Association for Computing Machinery, New York, NY, USA, 16–31. <https://doi.org/10.1145/3308561.3353774>
- [11] Sushant Kafle, Abraham Glasser, Sedeeq Al-khazraji, Larwan Berke, Matthew Seita, and Matt Huenerfauth. 2020. Artificial intelligence fairness in the context of accessibility research on intelligent systems for people who are deaf or hard of hearing. *SIGACCESS Access. Comput.*, 125, Article 4 (October 2019), 1 pages. <https://doi.org/10.1145/3386296.3386300>
- [12] “WFD AND WASLI STATEMENT ON USE OF SIGNING AVATARS Click here for International Sign Version.” Accessed: Nov. 06, 2025. [Online]. Available: <https://wasli.org/wp-content/uploads/2023/07/WFD-and-WASLI-Statement-on-Avatar-FINAL-14032018-Updated-14042018-1.pdf>
- [13] “Saving games,” *Godot Engine documentation*. https://docs.godotengine.org/en/stable/tutorials/io/saving_games.html
- [14] L. Jones, “Using Meta Quest Headsets in Education: 3 Powerful Ways Educators Can Get Started,” *Thinglink.com*, Oct. 27, 2025. <https://www.thinglink.com/blog/using-meta-quest-headsets-in-education-3-powerful-ways-educators-can-get-started/>
- [15] Y. Ding and J. Hao, “The Application of Educational Games Based on Virtual Reality Technology in Teaching English in Universities,” *Systems and Soft Computing*, p. 200380, Aug. 2025, doi: <https://doi.org/10.1016/j.sasc.2025.200380>.
- [16] Z. Zhang, “Innovative Application and User Experience of Virtual Reality Technology in Human-Computer Interaction,” *Highlights in Science, Engineering and Technology*, vol. 93, pp. 200–209, May 2024, doi: <https://doi.org/10.54097/mx8yf812>.

- [17] M. Golubski, "VR Music Learner Final Project Report," BSc dissertation, School of Computer Science, Technological University Dublin, Mar. 2022.
- [18] A. Botnari, "Sign Language Translator (SLT): Final Year Project," BSc dissertation, School of Computing, Department of Science and Health, Technological University Dublin, 2017.

A) Appendix A: System Model and Analysis

Interview Gathering:

B) Appendix B: Design

Details of design approach used

System elements, components, classes

C) Appendix C: Prompts Used with ChatGPT

Literature Review:

Prompt: "Could you suggest what I should write in this section, including relevant subheadings and topics for discussion, when structuring my literature review?"

Technologies Researched:

Prompt: "What should I write in this section, and what headings or subheadings would be appropriate for discussing the technologies I considered?"

System Analysis:

Prompt: "What are the important points to include in system analysis? Please advise on the main subheadings and structure for the section."

System Design:

Prompt: "Tell me what I should write here, listing suitable headings and subheadings for an effective system design section."

Prototype Development & Results:

Prompt: "Could you specify what needs to be included in the prototype and results section, as well as recommend potential subheadings and topics?"

Testing & Evaluation:

Prompt: "What should I cover while writing about testing and evaluation, and what subheadings or main headings would best organize this content?"

Issues and Future Work:

Prompt: "Suggest what to write about current issues and future improvements, including possible headings or subsections for clarity."

What are good diagrams to show for this report (aka class diagram, DFD, etc)

D) Appendix D: Additional Code Samples

```
1  extends Node
2
3  # Get References to Hand Pose Detectors
4  @onready var left_detector = $XR0rigin3D/LeftTrackedHand/HandPoseDetector
5  @onready var right_detector = $XR0rigin3D/RightTrackedHand/HandPoseDetector
6
7  # Reference to Label3D (SignText)
8  @onready var sign_text = $XR0rigin3D/ISLWall/SignText
9
10 var left_letter := ""
11 var right_letter := ""
12
13 func _ready() -> void:
14     >| # Connect signals from both detectors
15     >| left_detector.pose_started.connect(_on_left_pose_detected)
16     >| right_detector.pose_started.connect(_on_right_pose_detected)
17     >|
18     # Signal Handlers
19     >| func _on_left_pose_detected(pose_name: String) -> void:
20     >|     # Debug Message:
21     >|     print("Left Pose: ", pose_name)
22     >|     left_letter = pose_to_letter(pose_name)
23     >|     update_display()
24     >|
25     >| func _on_right_pose_detected(pose_name: String) -> void:
26     >|     # Debug Message:
27     >|     print("Right Pose: ", pose_name)
28     >|     right_letter = pose_to_letter(pose_name)
29     >|     update_display()
30     >|
31     # Pose -> Letter Mapping
32     >| func pose_to_letter(pose_name: String) -> String:
33     >|     # Debug message
34     >|     print("pose_to_letter() called with: ", pose_name)
35     >|     match pose_name:
36     >|         >| "Fist":
37     >|         >|     >| return "A"
38     >|         >| "Peace Sign":
39     >|         >|     >| return "V"
40     >|         >| "Point":
41     >|         >|     >| return "D"
42     >|         >| _:
43     >|         >|     >| return ""
44     >|         >|
45     # Update Text on Label
46     >| func update_display() -> void:
47     >|     sign_text.text = "Left hand = %s\nRight hand = %s" % [
48     >|         left_letter,
49     >|         right_letter
50     >|     ]
51
```

E) Appendix D: Additional Code Samples

E.1 Sign & Spell VR Prototype – Interview Summary

E.1.1 I Like

- Hand-tracking for sign language in VR feels engaging and modern for learners.
- Real-time visual feedback while fingerspelling helps users correct themselves instead of just copying videos.
- Having different practice modes (alphabet, words, names) makes practice more relevant and less repetitive.

E.1.2 I Wish

- Hand-tracking accuracy and consistency need improvement; missed gestures are frustrating.
- Users want clearer onboarding, calibration help, and simple step-by-step tutorials.
- More guidance when mistakes happen (for example, hints or example animations) would make learners feel more supported.

E.1.3 What If

- Users would like content beyond fingerspelling, including full ISL signs, everyday phrases, and basic ISL grammar.
- Social or multiplayer practice, plus progress tracking and rewards, could help keep learners motivated.
- Support for more VR headsets and some offline functionality would make the app easier to use in schools and areas with poor internet.

